
Python & Kafka Interview Q&A Handbook

Core & advanced Python, concurrency, Pandas, FastAPI/Django/Flask, Kafka internals, delivery semantics, failure handling and schema management

14 topics | 194 interview questions | 122 code examples

Companion to: [Interview_Prep_Tracker.xlsx](#)

How to use this handbook

Each topic starts with a short **overview** of what interviewers probe, then **must-know points** for quick revision, followed by numbered **interview questions with model answers** and a larger **worked example**. Question IDs (for example T07 - Q05) match the **Question Bank** sheet in the tracker, so you can mark each one as you master it.

Suggested routine: read the answer, close the PDF, explain it out loud in under two minutes, then type and run the code yourself. Mark the question *Confident* only when you can do all three without looking.

Contents

T01	Data Structures (List, Tuple, Dict, Set/FrozenSet) + collections	15 Q
T02	Comprehensions	12 Q
T03	Writing and Using Custom Decorators	13 Q
T04	Generators, Iterators and Iterables	13 Q
T05	Context Managers, Descriptors and Metaclasses	14 Q
T06	OOP in Depth (Multiple / Multilevel Inheritance)	14 Q
T07	Concurrency (threading / multiprocessing / asyncio)	14 Q
T08	Error Handling	14 Q
T09	Pandas for Data Handling	14 Q
T10	Web Frameworks: Django / Flask / FastAPI (FastAPI in depth)	14 Q
T11	Kafka Core: Topics, Partitions, Replication, Producers, Consumers	15 Q
T12	Designing Kafka Pipelines in Python and Delivery Semantics	14 Q
T13	Kafka Failure Handling: Retry Topics, DLQ, Consumer Idempotency	14 Q
T14	Schema Management in Kafka (Schema Registry, Avro/Protobuf/JSON Schema)	14 Q

T01 Data Structures (List, Tuple, Dict, Set/FrozenSet) + collections

Interviewers test whether you know the internal behaviour (mutability, hashing, time complexity) and can pick the right structure. The collections module (Counter, defaultdict, deque, namedtuple, OrderedDict, ChainMap) is where most 'clever' answers come from.

Must-know points

- list = dynamic array: $O(1)$ append/index, $O(n)$ insert(0)/pop(0)/**in**.
- tuple = immutable, hashable if all items are hashable, slightly smaller and faster than list.
- dict = hash table, insertion ordered since 3.7, $O(1)$ average get/set/delete.
- set = hash table of keys only, $O(1)$ membership; frozenset is the immutable/hashable version.
- deque gives $O(1)$ appends/pops at both ends; use it for queues and sliding windows.

Interview questions & answers

T01-Q01 What is the difference between a list and a tuple? When would you choose a tuple?

A list is mutable and a tuple is immutable. Because tuples cannot change, Python can store them more compactly, cache them as constants, and hash them (if all elements are hashable), so tuples can be dict keys or set members. Choose a tuple for fixed records (coordinates, DB rows, function returning multiple values) and when you need a hashable key; choose a list for a growing/shrinking collection of similar items.

```
import sys
print(sys.getsizeof([1, 2, 3])) # 88 (bigger: over-allocates for growth)
print(sys.getsizeof((1, 2, 3))) # 64
d = {(10, 20): "point"}        # tuple as dict key -> OK
# d[[10, 20]] = "x"             # TypeError: unhashable type: 'list'
```

T01-Q02 Is a tuple always hashable? Is a tuple truly immutable?

No. A tuple is only hashable if every element is hashable. The tuple itself is immutable (you cannot rebind its slots), but if it contains a mutable object such as a list, that inner object can still change. `hash((1, [2]))` raises `TypeError`.

```
t = (1, [2, 3])
t[1].append(4) # allowed -> (1, [2, 3, 4])
# t[1] = [9]   # TypeError: 'tuple' object does not support item assignment
# hash(t)      # TypeError: unhashable type: 'list'
```

T01-Q03 How is a Python dict implemented, and what is the time complexity of its operations?

A dict is an open-addressing hash table. Since CPython 3.6 it uses a compact layout: a sparse index array plus a dense entries array (hash, key, value), which saves memory and preserves insertion order (guaranteed by the language from 3.7). Lookup, insert and delete are $O(1)$ on average and $O(n)$ in the worst case (many hash collisions). The table resizes when it is about two-thirds full.

T01-Q04 What makes an object usable as a dict key or set element?

It must be hashable: it implements `__hash__` returning a value that never changes during its lifetime, and `__eq__` consistent with it ($a == b$ implies $\text{hash}(a) == \text{hash}(b)$). Built-in immutables (int, str, tuple of hashables, frozenset) qualify. A user class is hashable by identity by default; if you override `__eq__` without `__hash__`, Python sets `__hash__ = None` and instances become unhashable.

T01-Q05 Difference between set and frozenset? Give a real use of frozenset.

Both are unordered collections of unique hashable items with $O(1)$ membership. A set is mutable (add/remove); a frozenset is immutable and therefore hashable, so it can be a dict key or an element of another set. Typical use: caching results keyed by an unordered group, e.g. permissions or a set of tags.

```
cache = {}
perms = frozenset({"read", "write"})
cache[perms] = "editor"
print(cache[frozenset({"write", "read"})]) # 'editor' (order doesn't matter)
groups = {frozenset({1, 2}), frozenset({2, 1})}
print(len(groups)) # 1
```

T01-Q06 Explain shallow copy vs deep copy for nested lists and dicts.

A shallow copy (`list.copy()`, `dict.copy()`, slicing, `copy.copy`) creates a new outer container but reuses references to the inner objects. A deep copy (`copy.deepcopy`) recursively copies inner objects too. Mutating a nested object after a shallow copy is visible in both copies.

```
import copy
a = [[1, 2], [3, 4]]
s = a.copy(); d = copy.deepcopy(a)
a[0].append(99)
print(s) # [[1, 2, 99], [3, 4]] shared inner list
print(d) # [[1, 2], [3, 4]] independent
```

T01-Q07 What is the classic mutable default argument bug?

Default values are evaluated once, at function definition time. A mutable default (list/dict) is therefore shared across all calls. Use `None` as a sentinel and create the object inside the function.

```
def add(item, bucket=[]): # BAD
    bucket.append(item); return bucket
add(1); print(add(2)) # [1, 2] <- surprise

def add(item, bucket=None): # GOOD
    bucket = [] if bucket is None else bucket
    bucket.append(item); return bucket
```

T01-Q08 When would you use collections.Counter? Show a top-k example.

Counter is a dict subclass for counting hashable items. It supports `most_common(k)`, arithmetic (+, -, &, |) between counters, and returns 0 for missing keys instead of raising `KeyError`. Great for frequency analysis, anagram checks and log analysis.

```
from collections import Counter
words = "a b a c b a d".split()
c = Counter(words)
print(c.most_common(2)) # [('a', 3), ('b', 2)]
print(Counter("listen") == Counter("silent")) # True -> anagram
```

T01-Q09 defaultdict vs dict.setdefault vs dict.get — which and why?

`defaultdict(factory)` automatically creates a default value on first access of a missing key, which keeps grouping code clean. `setdefault(k, v)` does the same for a single call on a normal dict (but always constructs `v`). `get(k, default)` only reads and never inserts. Note: with `defaultdict` even a read like `d[k]` inserts the key, which can hide bugs.

```
from collections import defaultdict
orders = [("alice", 10), ("bob", 5), ("alice", 7)]
by_user = defaultdict(list)
for user, amt in orders:
    by_user[user].append(amt)
print(dict(by_user))  # {'alice': [10, 7], 'bob': [5]}
```

T01-Q10 Why use deque instead of list for a queue?

`list.pop(0)` and `list.insert(0, x)` are $O(n)$ because every element shifts. `deque` is a doubly-linked list of blocks with $O(1)$ append/pop on both ends. `deque(maxlen=n)` also gives a fixed-size ring buffer, handy for 'last N events' or sliding windows. Random indexing into the middle of a deque is $O(n)$.

```
from collections import deque
last5 = deque(maxlen=5)
for i in range(8):
    last5.append(i)
print(last5)  # deque([3, 4, 5, 6, 7], maxlen=5)
q = deque([1, 2]); q.appendleft(0); q.popleft()
```

T01-Q11 What are namedtuple and dataclass, and how do they differ?

`namedtuple` creates an immutable tuple subclass with named fields — lightweight, hashable, unpackable. `@dataclass` generates `__init__`, `__repr__`, `__eq__` for a normal class; it is mutable by default (`frozen=True` makes it immutable/hashable), supports type hints, defaults and methods, and `slots=True` (3.10+) for memory. Use `namedtuple` for simple records returned from functions; `dataclass` for richer domain objects.

```
from collections import namedtuple
from dataclasses import dataclass
Point = namedtuple("Point", "x y")
p = Point(1, 2); x, y = p

@dataclass(frozen=True, slots=True)
class Order:
    id: int
    amount: float = 0.0
```

T01-Q12 Is OrderedDict still useful now that dict preserves order?

Yes, in a few cases: `move_to_end()` and `popitem(last=False)` make it ideal for an LRU cache; equality between two `OrderedDicts` is order-sensitive (plain dict equality ignores order); and it communicates intent clearly.

```
from collections import OrderedDict
class LRU:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return -1
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)
```

T01-Q13 What is ChainMap used for?

ChainMap groups several dicts into one view and looks up keys in order, without copying. A classic use is layered configuration: CLI args override environment variables, which override defaults. Writes go to the first mapping only.

```
from collections import ChainMap
import os
defaults = {"env": "dev", "debug": False}
cli = {"debug": True}
cfg = ChainMap(cli, os.environ, defaults)
print(cfg["debug"], cfg["env"])  # True dev
```

T01-Q14 How do you remove duplicates from a list while keeping order?

`list(set(x))` loses order. Since dicts keep insertion order, `list(dict.fromkeys(x))` is the idiomatic $O(n)$ solution. For unhashable items you need a manual loop with a seen-set of some hashable projection.

```
items = [3, 1, 3, 2, 1]
print(list(dict.fromkeys(items)))  # [3, 1, 2]
```

T01-Q15 What does `is` vs `==` check, and why does `a is b` sometimes return True for small ints?

`==` compares values via `__eq__`; `is` compares identity (same object in memory). CPython caches small integers (-5 to 256) and interns some strings, so `is` may accidentally return True for them — never rely on that. Use `is` only for singletons like `None`, `True`, `False`, or sentinel objects.

Worked example: Sliding-window maximum with deque (common coding round problem)

```
from collections import deque

def max_sliding_window(nums, k):
    dq, out = deque(), []  # dq stores indices, values decreasing
    for i, n in enumerate(nums):
        while dq and nums[dq[-1]] < n:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:  # drop index outside window
            dq.popleft()
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out

print(max_sliding_window([1,3,-1,-3,5,3,6,7], 3))  # [3,3,5,5,6,7]
```

Worked example: Group anagrams using defaultdict + tuple keys

```
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[tuple(sorted(w))].append(w)  # tuple is hashable
    return list(groups.values())

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
```

T02 Comprehensions

Comprehensions are a readability and performance topic. Expect questions on list/dict/set comprehensions, generator expressions, nesting order, scope rules and when NOT to use them.

Must-know points

- Four forms: [list], {set}, {k: v dict}, (generator expression).
- Nested loops read left-to-right in the same order as normal for-loops.
- `x if c else y` before `for` transforms; `if c` after `for` filters.
- Comprehensions have their own scope (the loop variable does not leak in Python 3).
- Use a generator expression when you only iterate once or feed `sum/any/max` — it saves memory.

Interview questions & answers

T02-Q01 What are the four kinds of comprehensions in Python?

List `[x*x for x in it]`, set `{x % 3 for x in it}`, dict `{k: v for k, v in pairs}`, and generator expressions `(x*x for x in it)`. The first three build the whole collection in memory; the generator expression produces items lazily.

```
nums = range(6)
print([n*n for n in nums])           # [0,1,4,9,16,25]
print({n % 3 for n in nums})         # {0,1,2}
print({n: n*n for n in nums if n % 2}) # {1:1, 3:9, 5:25}
print(sum(n*n for n in nums))        # 55, no list created
```

T02-Q02 Why is a list comprehension usually faster than an equivalent for-loop with append?

The comprehension runs as a specialised bytecode loop using `LIST_APPEND`, avoiding the repeated attribute lookup and method call of `result.append` on each iteration. The gain is typically 10-30%. It is still $O(n)$; the win is constant-factor, and readability is the bigger reason to use it.

T02-Q03 How do you read a nested comprehension? Flatten a matrix.

Write the `for` clauses in the same order you would nest the loops. `[x for row in matrix for x in row]` equals `for row in matrix: for x in row: result.append(x)`.

```
matrix = [[1, 2, 3], [4, 5, 6]]
flat = [x for row in matrix for x in row]      # [1,2,3,4,5,6]
transposed = [[row[i] for row in matrix] for i in range(3)]
# [[1,4],[2,5],[3,6]]
```

T02-Q04 Where do `if` and `if-else` go in a comprehension?

A filtering `if` goes after the `for` and removes items. A conditional expression `a if cond else b` goes before the `for` and transforms every item. You can combine both.

```
nums = [1, 2, 3, 4, 5]
evens = [n for n in nums if n % 2 == 0]        # filter
labels = ["even" if n % 2 == 0 else "odd" for n in nums] # transform
```

T02-Q05 List comprehension vs generator expression — memory and behaviour?

A list comprehension builds all items immediately ($O(n)$ memory, reusable, supports len/indexing). A generator expression yields one item at a time ($O(1)$ memory), can only be consumed once, and is ideal for pipelines or large files.

```
import sys
lst = [i for i in range(1_000_000)]
gen = (i for i in range(1_000_000))
print(sys.getsizeof(lst))    # ~8 MB
print(sys.getsizeof(gen))    # ~200 bytes
```

T02-Q06 Does the loop variable of a comprehension leak into the enclosing scope?

Not in Python 3. Each comprehension runs in its own implicit function scope (inlined in 3.12 via PEP 709 but the variable still does not leak). In Python 2 list comprehensions did leak. Note that the walrus operator `:=` inside a comprehension deliberately binds in the enclosing scope.

```
x = "outer"
squares = [x*x for x in range(3)]
print(x)    # 'outer'
data = [3, 8, 1]
if any((big := n) > 5 for n in data):
    print(big) # 8 (walrus leaks on purpose)
```

T02-Q07 What is the late-binding closure trap with lambdas in comprehensions?

Closures capture variables, not values. Lambdas created in a loop all see the final value of the loop variable. Fix by binding the current value as a default argument.

```
fns = [lambda: i for i in range(3)]
print([f() for f in fns])    # [2, 2, 2]
fns = [lambda i=i: i for i in range(3)]
print([f() for f in fns])    # [0, 1, 2]
```

T02-Q08 How do you invert a dictionary with a comprehension? What if values repeat?

`{v: k for k, v in d.items()}` works when values are unique and hashable; for duplicate values later keys overwrite earlier ones. To keep all keys, group with `defaultdict(list)`.

```
d = {"a": 1, "b": 2, "c": 1}
print({v: k for k, v in d.items()}) # {1: 'c', 2: 'b'}
```

T02-Q09 When should you NOT use a comprehension?

When the logic has side effects (printing, writing to DB) — use a normal loop; when it needs more than two levels of nesting or complex conditions — readability suffers; when you need try/except per item; or when the result is huge and only iterated once — use a generator expression instead.

T02-Q10 How can you use a comprehension with a function that may fail for some items?

Comprehensions cannot contain try/except, so wrap the risky operation in a small helper that returns a sentinel, then filter it out.

```
def to_int(s):
    try: return int(s)
    except ValueError: return None
raw = ["1", "x", "3"]
nums = [n for s in raw if (n := to_int(s)) is not None] # [1, 3]
```


T02-Q11 How would you count word frequency using a dict comprehension, and what is the better way?

`{w: words.count(w) for w in set(words)}` works but is $O(n*k)$ because each `count()` scans the list. `collections.Counter(words)` is $O(n)$ and the idiomatic choice. Interviewers like to see that you spot the hidden quadratic cost.

T02-Q12 Can you build a dict from two lists using a comprehension?

Yes: `{k: v for k, v in zip(keys, values)}`, though `dict(zip(keys, values))` is simpler. `zip` stops at the shortest input; use `itertools.zip_longest` or `zip(..., strict=True)` (3.10+) to detect length mismatch.

```
keys, vals = ["a", "b"], [1, 2]
print(dict(zip(keys, vals, strict=True))) # {'a': 1, 'b': 2}
```

Worked example: Real-world: transform API records with nested comprehension

```
orders = [
    {"id": 1, "items": [{"sku": "A", "qty": 2}, {"sku": "B", "qty": 0}]},
    {"id": 2, "items": [{"sku": "A", "qty": 1}]}
]
# flatten non-zero line items
lines = [(o["id"], it["sku"], it["qty"])
         for o in orders for it in o["items"] if it["qty"] > 0]
# total qty per sku
skus = {sku for _, sku, _ in lines}
totals = {s: sum(q for _, sk, q in lines if sk == s) for s in skus}
print(lines) # [(1, 'A', 2), (2, 'A', 1)]
print(totals) # {'A': 3}
```

T03 Writing and Using Custom Decorators

Decorators test your understanding of first-class functions, closures and `*args/**kwargs`. Be ready to write one live: timing, retry, caching, auth and a decorator that accepts arguments.

Must-know points

- A decorator is a callable that takes a function and returns a (usually wrapped) function.
- `@deco` above `def f` is just `f = deco(f)`.
- Always use `functools.wraps` to preserve `__name__`, `__doc__`, `__wrapped__`.
- Decorator with arguments = a factory returning a decorator (three levels).
- Stacked decorators apply bottom-up and run top-down.

Interview questions & answers

T03-Q01 What is a decorator? Write the simplest possible one.

A decorator is a function that receives another function and returns a replacement, usually a wrapper that adds behaviour before/after calling the original. It relies on functions being first-class objects and on closures (the wrapper remembers `func`).

```
import functools
def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"calling {func.__name__} {args} {kwargs}")
        return func(*args, **kwargs)
    return wrapper

@log_calls
def add(a, b): return a + b
add(2, 3)
```

T03-Q02 Why do we use `functools.wraps`?

Without it the wrapper replaces the original function's metadata: `__name__` becomes 'wrapper', the docstring is lost, and introspection tools, logging, Sphinx, FastAPI/Flask routing and pytest may break. `wraps` copies `__name__`, `__qualname__`, `__doc__`, `__module__`, `__dict__` and sets `__wrapped__` so the original can be reached.

T03-Q03 How do you write a decorator that takes arguments? Write `@retry(times=3)`.

You need one extra level: an outer factory that receives the arguments and returns the real decorator.

```
import functools, time
def retry(times=3, delay=0.5, exceptions=(Exception,)):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    if attempt == times: raise
                    print(f"attempt {attempt} failed: {e}")
                    time.sleep(delay * 2 ** (attempt - 1)) # backoff
            return wrapper
        return decorator

@retry(times=3, exceptions=(ConnectionError,))
def call_api(): ...
```

T03-Q04 In what order are stacked decorators applied?

Bottom-up at definition time: `@a @b def f` becomes `f = a(b(f))`. At call time the outermost wrapper (a) runs first, then b, then f — like layers of an onion.

```
@bold      # applied second, runs first
@italic    # applied first
def text(): return "hi"
# text() -> bold(italic(text))() -> '<b><i>hi</i></b>'
```

T03-Q05 How do you write a decorator that works with and without parentheses?

Check whether the first positional argument is callable. If it is, the decorator was used bare; otherwise it was called with options and must return the decorator.

```
import functools
def timer(_func=None, *, unit="ms"):
    def deco(func):
        @functools.wraps(func)
        def wrapper(*a, **k):
            import time; t = time.perf_counter()
            try: return func(*a, **k)
            finally:
                ms = (time.perf_counter() - t) * 1000
                print(f"{func.__name__}: {ms:.2f}{unit}")
        return wrapper
    return deco(_func) if callable(_func) else deco

@timer
def f(): ...
@timer(unit="ms")
def g(): ...
```

T03-Q06 How do you write a class-based decorator? When is it useful?

Implement `__init__(self, func)` and `__call__(self, *args, **kwargs)`. Useful when the decorator needs state (call counts, caches, rate-limit buckets). Use `functools.update_wrapper(self, func)` for metadata.

```
import functools
class CountCalls:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func, self.calls = func, 0
    def __call__(self, *a, **k):
        self.calls += 1
        return self.func(*a, **k)

@CountCalls
def ping(): return "pong"
ping(); ping(); print(ping.calls) # 2
```

T03-Q07 How do decorators work on methods? What about the `self` argument?

A function-based decorator works on methods unchanged because `self` just arrives as the first element of `*args`. A class-based decorator does NOT bind automatically, because instances of your class are not descriptors — you must implement `__get__` (or use `functools.partial`) so `self` is passed.

T03-Q08 What does `functools.lru_cache` do and what are its caveats?

It memoizes a function's results in a dict keyed by the arguments, evicting least-recently-used entries past `maxsize`. Caveats: all arguments must be hashable; it holds strong references (memory growth, can keep objects alive); on instance methods it caches per `self` and keeps instances alive — prefer `functools.cached_property` or a per-instance cache; it is thread-safe but not process-shared.

```
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
print(fib(100), fib.cache_info())
```

T03-Q09 How do you write a decorator for async functions?

The wrapper itself must be `async def` and must `await` the original. A single decorator can support both by checking `inspect.iscoroutinefunction(func)`.

```
import functools, inspect, time
def timed(func):
    if inspect.iscoroutinefunction(func):
        @functools.wraps(func)
        async def aw(*a, **k):
            t = time.perf_counter()
            try: return await func(*a, **k)
            finally: print(func.__name__, time.perf_counter() - t)
        return aw
    @functools.wraps(func)
    def w(*a, **k):
        t = time.perf_counter()
        try: return func(*a, **k)
        finally: print(func.__name__, time.perf_counter() - t)
    return w
```

T03-Q10 Can you decorate a class? Give an example.

Yes: a class decorator receives the class object and returns it (modified) or a replacement. `@dataclass` and `@functools.total_ordering` are examples. A common custom one is a registry or singleton.

```

REGISTRY = {}
def register(cls):
    REGISTRY[cls.__name__] = cls
    return cls

@register
class CsvParser: ...
print(REGISTRY) # {'CsvParser': <class ...>}

```

T03-Q11 Write an authorization decorator like those used in Flask/FastAPI apps.

The decorator inspects the current user (from a kwarg, context var or request) and raises before calling the view if the permission is missing.

```

import functools
class Forbidden(Exception): pass
def require_role(role):
    def deco(func):
        @functools.wraps(func)
        def wrapper(user, *a, **k):
            if role not in user.get("roles", []):
                raise Forbidden(f"{role} required")
            return func(user, *a, **k)
        return wrapper
    return deco

@require_role("admin")
def delete_user(user, uid): return f"deleted {uid}"
print(delete_user({"roles": ["admin"]}, 7))

```

T03-Q12 What are the built-in decorators you should know?

`@staticmethod`, `@classmethod`, `@property` (plus `.setter/.deleter`), `@functools.wraps`, `@functools.lru_cache/@cache`, `@functools.cached_property`, `@functools singledispatch`, `@functools.total_ordering`, `@dataclasses.dataclass`, `@contextlib.contextmanager`, `@abc.abstractmethod`, and `@typing.overload`.

T03-Q13 How do you unit-test a decorated function or bypass the decorator?

Test the decorator separately with a dummy function. To reach the original, use `func.__wrapped__` (set by `functools.wraps`). With `pytest` you can also monkeypatch `time.sleep` in retry decorators so tests run fast.

Worked example: Production-style rate limiter decorator (token bucket, thread-safe)

```

import functools, threading, time

def rate_limit(calls: int, per: float):
    lock = threading.Lock()
    tokens, last = [calls], [time.monotonic()]
    def deco(func):
        @functools.wraps(func)
        def wrapper(*a, **k):
            with lock:
                now = time.monotonic()
                tokens[0] = min(calls, tokens[0] + (now - last[0]) * calls / per)
                last[0] = now
                if tokens[0] < 1:
                    raise RuntimeError("rate limit exceeded")
                tokens[0] -= 1
            return func(*a, **k)
        return wrapper
    return deco

@rate_limit(calls=5, per=1.0)
def send_sms(to): return f"sent to {to}"

```

T04 Generators, Iterators and Iterables

Know the iterator protocol precisely, how `yield` suspends a frame, lazy pipelines, `yield from`, generator methods (`send/throw/close`) and `itertools`. These are favourite questions for data-processing and Kafka-consumer style roles.

Must-know points

- Iterable: has `__iter__` returning an iterator (list, dict, file, range).
- Iterator: has `__next__` and `__iter__` returning self; raises `StopIteration` when done.
- Generator function: contains `yield`; calling it returns a generator (an iterator).
- Generators are lazy, single-pass, and keep local state between yields.
- `yield from` delegates to a sub-iterator and forwards `send/throw`.

Interview questions & answers

T04-Q01 What is the difference between an iterable and an iterator?

An iterable can produce an iterator: it implements `__iter__` (or the old `__getitem__` protocol). An iterator produces values one at a time via `__next__` and returns itself from `__iter__`. A list is iterable but not an iterator — you can loop over it many times; an iterator is exhausted after one pass.

```
nums = [1, 2]
it = iter(nums)          # list -> list_iterator
print(next(it), next(it))
# next(it) -> StopIteration
print(iter(it) is it)    # True: iterators return themselves
```

T04-Q02 How does a for-loop work internally?

`for x in obj` calls `iter(obj)` once, then repeatedly calls `next()` on the iterator, binding the result to `x`, until `StopIteration` is raised, which the loop catches silently.

```
it = iter([10, 20])
while True:
    try:
        x = next(it)
    except StopIteration:
        break
    print(x)
```

T04-Q03 Write a custom iterator class.

Implement `__iter__` returning self and `__next__` returning the next value or raising `StopIteration`.

```
class Countdown:
    def __init__(self, start): self.cur = start
    def __iter__(self): return self
    def __next__(self):
        if self.cur <= 0: raise StopIteration
        self.cur -= 1
        return self.cur + 1
print(list(Countdown(3))) # [3, 2, 1]
```

T04-Q04 What is a generator and how does `yield` work?

A generator function contains `yield`. Calling it does not run the body; it returns a generator object. Each `next()` runs until the next `yield`, returns that value, and freezes the frame (locals, instruction pointer). When the function returns, `StopIteration` is raised. This gives iterator behaviour with far less code.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
g = countdown(3)
print(next(g), next(g), list(g))    # 3 2 [1]
```

T04-Q05 Why are generators memory-efficient? Give a real-world example.

They produce values lazily and hold only the current state, not the whole sequence. Reading a 20 GB log file line by line with a generator uses constant memory.

```
def read_errors(path):
    with open(path) as f:
        for line in f:
            if "ERROR" in line:
                yield line.rstrip()

# for err in read_errors("app.log"): process(err)
```

T04-Q06 What does `yield from` do?

It delegates to a sub-iterable: yields every item from it, passes values sent with `send()` and exceptions from `throw()` through to the sub-generator, and evaluates to the sub-generator's return value. It simplifies recursive generators.

```
def flatten(items):
    for x in items:
        if isinstance(x, list):
            yield from flatten(x)
        else:
            yield x
print(list(flatten([1, [2, [3, 4]], 5])))    # [1,2,3,4,5]
```

T04-Q07 Explain `generator.send()`, `throw()` and `close()`.

`send(v)` resumes the generator and makes the paused `yield` expression evaluate to `v` (the generator must first be primed with `next()` or `send(None)`). `throw(exc)` raises an exception at the paused yield. `close()` raises `GeneratorExit` inside it so `finally` blocks run.

```
def running_avg():
    total = count = 0
    avg = None
    while True:
        value = yield avg
        total += value; count += 1
        avg = total / count
g = running_avg(); next(g)    # prime
print(g.send(10), g.send(20))    # 10.0 15.0
```

T04-Q08 What happens if you iterate a generator twice?

The second loop gets nothing — the generator is exhausted and immediately raises `StopIteration`. If you need multiple passes, materialise it to a list, call the generator function again, or use `itertools.tee` (which buffers).

T04-Q09 Generator expression vs generator function?

A generator expression (`f(x) for x in xs`) is a concise one-liner for simple map/filter. A generator function is needed for multiple yields, complex state, try/finally cleanup, or send/throw handling.

T04-Q10 Build a lazy data pipeline with generators.

Chain small generator stages; each pulls items from the previous one only when needed. This mirrors stream processing (Kafka consumer -> parse -> filter -> sink).

```
import json
def read(lines):
    for line in lines: yield line
def parse(lines):
    for l in lines:
        try: yield json.loads(l)
        except json.JSONDecodeError: continue
def only_paid(events):
    return (e for e in events if e.get("status") == "PAID")

raw = ['{"id":1,"status":"PAID"}', 'bad', '{"id":2,"status":"NEW"}']
for e in only_paid(parse(read(raw))):
    print(e)  # {'id': 1, 'status': 'PAID'}
```

T04-Q11 Which itertools functions do you use most?

`chain` (concatenate iterables), `islice` (slice an iterator lazily), `groupby` (group consecutive equal keys — sort first), `count/cycle/repeat`, `product/permutations/combinations`, `accumulate` (running totals), `tee`, `zip_longest`, and `batched` (3.12) for chunking.

```
from itertools import islice, groupby, accumulate
print(list(islice(range(100), 5)))  # first 5 lazily
data = sorted(["apple", "avocado", "banana"], key=lambda s: s[0])
print({k: list(g) for k, g in groupby(data, key=lambda s: s[0])})
print(list(accumulate([1, 2, 3, 4])))  # [1,3,6,10]
```

T04-Q12 How do you read a large iterable in fixed-size batches?

Use `itertools.batched` in Python 3.12+, or build it with `islice`. This is exactly how you would batch records before a bulk DB insert or a Kafka produce loop.

```
from itertools import islice
def batched(iterable, n):
    it = iter(iterable)
    while chunk := list(islice(it, n)):
        yield chunk
print(list(batched(range(7), 3)))  # [[0,1,2],[3,4,5],[6]]
```


T04-Q13 What is the difference between `return` inside a generator and `yield`?

`yield` produces a value and pauses. `return value` ends the generator; the value is attached to the `StopIteration` exception (`e.value`) and becomes the result of a `yield from` expression. A bare `return` just stops iteration.

Worked example: Infinite generator + `islice`: paginated API fetcher

```
from itertools import islice

def fetch_pages(client, page_size=100):
    page = 1
    while True:
        rows = client.get("/orders", params={"page": page, "size": page_size})
        if not rows:
            return # ends the generator
        yield from rows
        page += 1

# Take only first 250 orders without loading everything:
# first_250 = list(islice(fetch_pages(client), 250))
```

T05 Context Managers, Descriptors and Metaclasses

These are the 'advanced Python' signals. Context managers are practical (resources, transactions); descriptors explain how property/methods actually work; metaclasses show you understand how classes are created (ORMs, plugin registries).

Must-know points

- `with` calls `__enter__`, then always `__exit__(exc_type, exc, tb)`; returning `True` suppresses the exception.
- `@contextlib.contextmanager` turns a generator into a context manager.
- Descriptor = object with `__get__`/`__set__`/`__delete__` stored on a class.
- Data descriptors (define `__set__`) beat instance `__dict__`; non-data descriptors don't.
- Metaclass = the class of a class; `type` is the default. Prefer `__init_subclass__` or class decorators when possible.

Interview questions & answers

T05-Q01 What is a context manager and what problem does it solve?

An object that defines setup and teardown around a block via `__enter__` and `__exit__`. It guarantees cleanup (closing files, releasing locks, committing/rolling back transactions) even if an exception occurs, replacing verbose `try/finally` code.

T05-Q02 Write a class-based context manager for a database transaction.

`__exit__` receives the exception info. Commit if no exception, otherwise roll back. Return `False` (or `None`) so the exception propagates.

```
class Transaction:
    def __init__(self, conn): self.conn = conn
    def __enter__(self):
        self.cur = self.conn.cursor()
        return self.cur
    def __exit__(self, exc_type, exc, tb):
        if exc_type is None:
            self.conn.commit()
        else:
            self.conn.rollback()
        self.cur.close()
        return False          # don't swallow the exception

# with Transaction(conn) as cur:
#     cur.execute("UPDATE accounts SET bal = bal - 100 WHERE id = 1")
```

T05-Q03 How do you write a context manager using contextlib?

Decorate a generator with `@contextmanager`. Code before `yield` is `__enter__`, the yielded value is bound by `as`, and code after `yield` (put it in `finally`) is `__exit__`.

```
from contextlib import contextmanager
import time
@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f"{label}: {time.perf_counter() - start:.3f}s")

with timer("load"):
    sum(range(10**6))
```

T05-Q04 How can a context manager suppress an exception?

If `__exit__` returns a truthy value the exception is swallowed. `contextlib.suppress(*exceptions)` does this for you. Use sparingly — silent failures are hard to debug.

```
from contextlib import suppress
import os
with suppress(FileNotFoundError):
    os.remove("temp.txt")
```

T05-Q05 What other contextlib utilities are useful?

`ExitStack` (manage a dynamic number of context managers), `suppress`, `closing` (call `close()` on exit), `nullcontext` (a no-op placeholder), `redirect_stdout`, `asynccontextmanager` and `AsyncExitStack` for async code, and `chdir` (3.11).

```
from contextlib import ExitStack
files = ["a.txt", "b.txt"]
# with ExitStack() as stack:
#     handles = [stack.enter_context(open(f)) for f in files]
#     ... all closed automatically ...
```

T05-Q06 What is a descriptor?

Any object that defines `__get__`, `__set__` or `__delete__` and is stored as a class attribute. When you access that attribute on an instance, Python calls the descriptor's methods instead of returning the object. `property`, `staticmethod`, `classmethod`, functions (which become bound methods) and ORM fields are all descriptors.

T05-Q07 Data descriptor vs non-data descriptor — how does lookup precedence work?

A data descriptor defines `__set__` or `__delete__`; a non-data descriptor defines only `__get__`. Attribute lookup order for `obj.x`: 1) data descriptor on the type, 2) instance `__dict__`, 3) non-data descriptor or plain class attribute, 4) `__getattr__`. That is why you can shadow a method on an instance but cannot shadow a property.

T05-Q08 Write a reusable validation descriptor.

Use `__set_name__` (3.6+) to learn the attribute name, store the value in the instance dict under that name, and validate in `__set__`.

```

class Positive:
    def __set_name__(self, owner, name):
        self.name = "_" + name
    def __get__(self, obj, objtype=None):
        if obj is None: return self
        return getattr(obj, self.name)
    def __set__(self, obj, value):
        if not isinstance(value, (int, float)) or value <= 0:
            raise ValueError(f"{self.name[1:]} must be > 0")
        setattr(obj, self.name, value)

class Order:
    qty = Positive(); price = Positive()
    def __init__(self, qty, price): self.qty, self.price = qty, price
Order(2, 9.5)
# Order(0, 1) -> ValueError: qty must be > 0

```

T05-Q09 How is @property related to descriptors?

property is a built-in data descriptor class. **@property** creates a property object whose **__get__** calls your getter; **.setter** returns a new property with **__set__** wired to the setter. Because it is a data descriptor, it takes precedence over the instance dict.

T05-Q10 What is a metaclass?

A metaclass is the class of a class — it controls how class objects are created. **type** is the default metaclass: **class A: pass** is roughly **A = type('A', (), {})**. A custom metaclass subclasses **type** and overrides **__new__**/**__init__**/**__call__** to inspect or modify classes at creation time.

```

Dog = type("Dog", (), {"speak": lambda self: "woof"})
print(Dog().speak(), type(Dog))  # woof <class 'type'>

```

T05-Q11 Write a metaclass that enforces a rule on subclasses.

Override **__new__** to validate the class namespace before the class is created.

```

class RequireRun(type):
    def __new__(mcs, name, bases, ns):
        if bases and "run" not in ns:
            raise TypeError(f"{name} must define run()")
        return super().__new__(mcs, name, bases, ns)

class Job(metaclass=RequireRun): pass
class EmailJob(Job):
    def run(self): ...
# class BadJob(Job): pass -> TypeError

```

T05-Q12 Implement a Singleton using a metaclass. Is it a good idea?

Override **__call__** on the metaclass so repeated instantiation returns the cached instance. It works, but singletons are global state and hurt testability; in Python a module-level instance or dependency injection is usually better.

```

class Singleton(type):
    _inst = {}
    def __call__(cls, *a, **k):
        if cls not in cls._inst:
            cls._inst[cls] = super().__call__(*a, **k)
        return cls._inst[cls]

class Config(metaclass=Singleton): pass
print(Config() is Config())  # True

```

T05-Q13 What are alternatives to metaclasses?

`__init_subclass__` (hook called on the parent whenever a subclass is created — perfect for registries/validation), class decorators, `__set_name__` on descriptors, and `abc.ABC` for enforcing abstract methods. Tim Peters' rule: if you wonder whether you need a metaclass, you don't.

```
class Plugin:
    registry = {}
    def __init_subclass__(cls, key=None, **kw):
        super().__init_subclass__(**kw)
        Plugin.registry[key or cls.__name__] = cls

class Csv(Plugin, key="csv"): pass
print(Plugin.registry)  # {'csv': <class 'Csv'>}
```

T05-Q14 Where are metaclasses used in real frameworks?

Django's `ModelBase` collects Field descriptors into `_meta` and builds the ORM mapping; SQLAlchemy's declarative base; `abc.ABCMeta` for abstract classes; `enum.EnumMeta`; Pydantic v1's `ModelMetaclass` (v2 still uses a metaclass to collect fields). They all run code once at class-definition time to build metadata.

Worked example: Async context manager for a Kafka-style client resource

```
from contextlib import asynccontextmanager
import asyncio

class FakeProducer:
    async def start(self): print("connected")
    async def stop(self): print("flushed & closed")
    async def send(self, topic, v): print("sent", topic, v)

@asynccontextmanager
async def producer():
    p = FakeProducer()
    await p.start()
    try:
        yield p
    finally:
        await p.stop()  # always runs, even on error

async def main():
    async with producer() as p:
        await p.send("orders", b"hello")

asyncio.run(main())
```

T06 OOP in Depth (Multiple / Multilevel Inheritance)

Expect the four pillars, dunder methods, class vs static methods, abstract classes, and above all MRO (C3 linearisation), `super()` in cooperative multiple inheritance, and the diamond problem.

Must-know points

- Pillars: encapsulation, abstraction, inheritance, polymorphism.
- MRO = order Python searches classes; view with `Class.__mro__` / `Class.mro()`.
- `super()` goes to the NEXT class in the MRO, not necessarily the parent.
- Cooperative multiple inheritance: every class calls `super()` and accepts `**kwargs`.
- Prefer composition and mixins over deep hierarchies.

Interview questions & answers

T06-Q01 Explain the four pillars of OOP with Python examples.

Encapsulation: bundle data and behaviour, hide internals (by convention `_x`, name-mangling `__x`, properties). Abstraction: expose a simple interface, hide detail (`abc.ABC`). Inheritance: reuse and specialise behaviour from a base class. Polymorphism: different classes respond to the same interface (duck typing, method overriding, operator overloading).

T06-Q02 What is the difference between multilevel and multiple inheritance?

Multilevel: a chain, C inherits B which inherits A (grandparent relation). Multiple: a class inherits from two or more bases at once, `class C(A, B)`. Python supports both; multiple inheritance introduces the diamond problem, which Python resolves with MRO.

```
class Animal:
    def breathe(self): return "breathing"
class Mammal(Animal): pass          # multilevel: Animal -> Mammal -> Dog
class Dog(Mammal): pass

class Flyer:
    def move(self): return "fly"
class Swimmer:
    def move(self): return "swim"
class Duck(Flyer, Swimmer): pass    # multiple
print(Dog().breathe(), Duck().move()) # breathing fly
```

T06-Q03 What is the MRO and how does C3 linearisation work?

Method Resolution Order is the sequence of classes Python searches for an attribute. C3 guarantees: a class comes before its parents, parents keep the order they are listed in, and the order is consistent (monotonic) for every class. If no consistent order exists, Python raises `TypeError` at class creation.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
print([k.__name__ for k in D.__mro__]) # ['D', 'B', 'C', 'A', 'object']
```

T06-Q04 Explain the diamond problem and how Python solves it.

In a diamond (D inherits B and C, both inherit A) a naive approach would call A's method twice or ambiguously. Python's MRO lists A only once, after both B and C, and `super()` walks that list, so each class's method runs exactly once.

```
class A:
    def hello(self): print("A")
class B(A):
    def hello(self): print("B"); super().hello()
class C(A):
    def hello(self): print("C"); super().hello()
class D(B, C):
    def hello(self): print("D"); super().hello()
D().hello()      # D B C A (A runs once)
```

T06-Q05 What exactly does `super()` do?

`super()` returns a proxy that looks up the method in the class that comes AFTER the current class in the MRO of the instance's type (`type(self).__mro__`). In the diamond above, `super()` inside B calls C.hello — not A — when the object is a D. That is why it is called cooperative.

T06-Q06 How do you design cooperative `__init__` methods for multiple inheritance?

Every class accepts `**kwargs`, pops the arguments it needs, and calls `super().__init__(**kwargs)`. The chain terminates at `object`, which accepts no extra args.

```
class Base:
    def __init__(self, **kw): super().__init__(**kw)
class Timestamped(Base):
    def __init__(self, created=None, **kw):
        self.created = created; super().__init__(**kw)
class Named(Base):
    def __init__(self, name="", **kw):
        self.name = name; super().__init__(**kw)
class Product(Timestamped, Named): pass
p = Product(name="pen", created="2026-10-01")
print(p.name, p.created)
```

T06-Q07 What is a mixin? Give an example.

A small class that adds one focused capability and is not meant to stand alone, combined through multiple inheritance. Django's `LoginRequiredMixin` is a classic. Place mixins to the left of the main base so their methods come first in the MRO.

```
import json
class JsonMixin:
    def to_json(self): return json.dumps(self.__dict__)
class ReprMixin:
    def __repr__(self): return f"{type(self).__name__}({self.__dict__})"
class User(JsonMixin, ReprMixin):
    def __init__(self, name): self.name = name
print(User("ravi").to_json())
```

T06-Q08 Difference between instance method, @classmethod and @staticmethod?

Instance methods receive `self` and work on instance state. Class methods receive `cls` and are used for alternative constructors and class-level state; they respect subclassing. Static methods receive neither and are plain functions namespaced in the class.

```
class Date:
    def __init__(self, y, m, d): self.y, self.m, self.d = y, m, d
    @classmethod
    def from_str(cls, s): return cls(*map(int, s.split("-")))
    @staticmethod
    def is_leap(y): return y % 4 == 0 and (y % 100 != 0 or y % 400 == 0)
print(Date.from_str("2026-09-30").y, Date.is_leap(2028))
```

T06-Q09 How do you create an abstract class in Python?

Inherit from `abc.ABC` and mark methods with `@abstractmethod`. The class cannot be instantiated until a subclass implements all abstract methods. For structural (duck) typing without inheritance, use `typing.Protocol`.

```
from abc import ABC, abstractmethod
class PaymentGateway(ABC):
    @abstractmethod
    def charge(self, amount): ...
class Razorpay(PaymentGateway):
    def charge(self, amount): return f"charged {amount}"
# PaymentGateway() -> TypeError: Can't instantiate abstract class
```

T06-Q10 How does Python implement encapsulation? What is name mangling?

Python has no true private members. `_x` is a convention for 'internal'. `__x` (two leading underscores) triggers name mangling to `__ClassName__x` to avoid accidental clashes in subclasses — not for security. Use `@property` to control access with validation.

T06-Q11 What are dunder (magic) methods? Implement a few.

Special methods Python calls for built-in operations: `__init__`, `__repr__`/`__str__`, `__eq__`/`__hash__`, `__lt__`, `__len__`, `__getitem__`, `__iter__`, `__contains__`, `__add__`, `__call__`, `__enter__`/`__exit__`. They enable operator overloading and protocol support.

```
class Money:
    def __init__(self, amt): self.amt = amt
    def __add__(self, other): return Money(self.amt + other.amt)
    def __eq__(self, other): return isinstance(other, Money) and self.amt == other.amt
    def __hash__(self): return hash(self.amt)
    def __repr__(self): return f"Money({self.amt})"
print(Money(5) + Money(7))    # Money(12)
```

T06-Q12 Composition vs inheritance — which do you prefer and why?

Inheritance models 'is-a' and couples the child to the parent's implementation; deep hierarchies become fragile. Composition models 'has-a': the object holds collaborators and delegates to them, which is more flexible, testable (inject mocks) and follows 'favour composition over inheritance'. Use inheritance for true subtypes and small mixins.

T06-Q13 What is `__slots__` and when would you use it?

Declaring `__slots__ = ('x', 'y')` replaces the per-instance `__dict__` with fixed slots, cutting memory (often 40-50%) and making attribute access slightly faster. Useful when creating millions of small objects. Trade-offs: no dynamic attributes, and multiple inheritance with non-empty slots in several bases is restricted.

T06-Q14 What is method overriding vs overloading in Python?

Overriding: a subclass redefines a parent method (runtime polymorphism). Python has no signature-based overloading — the last definition wins. You emulate it with default/variadic arguments, or `functools.singledispatch` / `singledispatchmethod` to dispatch on argument type; `typing.overload` is only for type checkers.

Worked example: Putting it together: abstract base + mixins + cooperative super()

```
from abc import ABC, abstractmethod
import logging

class LoggingMixin:
    def log(self, msg): logging.getLogger(type(self).__name__).info(msg)

class RetryMixin:
    max_retries = 3
    def run_with_retry(self, *a):
        for i in range(self.max_retries):
            try: return self.process(*a)
            except Exception as e: self.log(f"retry {i+1}: {e}")
        raise RuntimeError("gave up")

class Handler(ABC):
    @abstractmethod
    def process(self, event): ...

class OrderHandler(LoggingMixin, RetryMixin, Handler):
    def process(self, event):
        self.log(f"processing {event}")
        return event["id"]

print(OrderHandler.__mro__)
print(OrderHandler().run_with_retry({"id": 42}))
```

T07 Concurrency (threading / multiprocessing / asyncio)

The central idea is the GIL and matching the tool to the workload: threads for blocking I/O, processes for CPU-bound work, asyncio for high-concurrency I/O. Expect questions on race conditions, locks, executors and event loops.

Must-know points

- GIL: only one thread runs Python bytecode at a time per process (CPython).
- I/O-bound -> threading or asyncio; CPU-bound -> multiprocessing / ProcessPoolExecutor.
- concurrent.futures gives one API for thread and process pools.
- asyncio = single-threaded cooperative multitasking; never block the event loop.
- Python 3.13 offers an experimental free-threaded (no-GIL) build; 3.12+ has per-interpreter GIL.

Interview questions & answers

T07-Q01 What is the GIL and how does it affect concurrency?

The Global Interpreter Lock is a mutex in CPython that allows only one thread to execute Python bytecode at a time. It simplifies memory management (reference counting). Consequence: threads don't speed up CPU-bound Python code, but they do help I/O-bound code because the GIL is released during blocking I/O and in many C extensions (NumPy, hashlib, compression). Python 3.13 introduced an optional free-threaded build (PEP 703) that removes it experimentally.

T07-Q02 Concurrency vs parallelism?

Concurrency is dealing with many tasks at once by interleaving them (one cook switching between dishes). Parallelism is executing tasks literally at the same time on multiple cores (many cooks). asyncio and threads give concurrency; multiprocessing gives parallelism.

T07-Q03 How do you choose between threading, multiprocessing and asyncio?

CPU-bound (image processing, heavy computation): multiprocessing/ProcessPoolExecutor to use all cores.
I/O-bound with a moderate number of tasks or blocking libraries (requests, JDBC-style DB drivers): threading/ThreadPoolExecutor.
I/O-bound with thousands of connections and async-capable libraries (aiohttp, asyncpg, aiokafka): asyncio.

T07-Q04 Show ThreadPoolExecutor for concurrent HTTP calls.

`concurrent.futures` manages a pool; `map` keeps order, `submit` + `as_completed` processes results as they finish.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time
def fetch(url):
    time.sleep(1)          # simulate blocking I/O
    return url, 200
urls = [f"https://api/{i}" for i in range(10)]
start = time.time()
with ThreadPoolExecutor(max_workers=10) as ex:
    futures = [ex.submit(fetch, u) for u in urls]
    for f in as_completed(futures):
        print(f.result())
print(f"{time.time() - start:.1f}s")  # ~1s, not 10s
```

T07-Q05 Show multiprocessing for CPU-bound work. Why the `__main__` guard?

Each process has its own interpreter and GIL. On Windows/macOS the default start method is 'spawn', which re-imports your module in children; without `if __name__ == '__main__':` the pool creation would run recursively. Arguments and results are pickled, so functions must be top-level.

```
from concurrent.futures import ProcessPoolExecutor
def cpu_heavy(n):
    return sum(i * i for i in range(n))
if __name__ == "__main__":
    with ProcessPoolExecutor() as ex:
        print(list(ex.map(cpu_heavy, [10**7] * 4)))
```

T07-Q06 What is a race condition? Show one and fix it.

When the result depends on the timing of threads accessing shared state. `counter += 1` is read-modify-write and not atomic, so updates can be lost. Protect it with a Lock (or avoid shared state with queues).

```
import threading
counter = 0
lock = threading.Lock()
def work():
    global counter
    for _ in range(100_000):
        with lock:
            counter += 1
            # remove the lock -> result may be < 400000
ts = [threading.Thread(target=work) for _ in range(4)]
[t.start() for t in ts]; [t.join() for t in ts]
print(counter) # 400000
```

T07-Q07 Name the threading synchronization primitives.

Lock (mutual exclusion), **RLock** (re-entrant, same thread can acquire again), **Semaphore/BoundedSemaphore** (limit concurrent access, e.g. max 5 DB connections), **Event** (one thread signals others), **Condition** (wait for a state change), **Barrier** (wait until N threads arrive), and thread-safe **queue.Queue** for producer-consumer.

T07-Q08 What is a deadlock and how do you avoid it?

Two or more threads each hold a lock the other needs and wait forever. Avoid by always acquiring locks in a consistent global order, using timeouts (`lock.acquire(timeout=...)`), holding locks for short sections, using RLock for re-entrant code, or avoiding shared locks with message passing (queues).

T07-Q09 How does asyncio work? Explain event loop, coroutine, task.

A coroutine (`async def`) is a function that can pause at `await`. The event loop runs in a single thread and schedules coroutines: when one awaits I/O it yields control and the loop runs another that is ready. A Task wraps a coroutine and schedules it to run concurrently; `asyncio.gather` or `TaskGroup` (3.11) runs many at once.

```
import asyncio
async def fetch(i):
    await asyncio.sleep(1) # non-blocking wait
    return i
async def main():
    async with asyncio.TaskGroup() as tg: # 3.11+
        tasks = [tg.create_task(fetch(i)) for i in range(100)]
    print(sum(t.result() for t in tasks)) # ~1s total
asyncio.run(main())
```

T07-Q10 What happens if you call `time.sleep()` or a blocking library inside async code?

It blocks the whole event loop thread, so every other coroutine freezes. Use async equivalents (`asyncio.sleep`, `aiohttp`, `asyncpg`) or offload blocking calls to a thread with `await asyncio.to_thread(func, ...)` or `loop.run_in_executor`.

```
import asyncio, time
def legacy_blocking(): time.sleep(2); return "done"
async def main():
    result = await asyncio.to_thread(legacy_blocking)
    print(result)
asyncio.run(main())
```

T07-Q11 How do you limit concurrency in asyncio (e.g. max 10 requests at a time)?

Use `asyncio.Semaphore`. Each task acquires it before doing I/O, so at most N run concurrently while the rest wait.

```
import asyncio
sem = asyncio.Semaphore(10)
async def call(i):
    async with sem:
        await asyncio.sleep(0.5)
        return i
async def main():
    return await asyncio.gather(*(call(i) for i in range(100)))
asyncio.run(main())
```

T07-Q12 How do you share data between processes?

Processes don't share memory by default. Options: `multiprocessing.Queue/Pipe` (message passing, pickled), `Value/Array` (shared ctypes memory with locks), `multiprocessing.shared_memory` (3.8, zero-copy buffers, good with NumPy), a `Manager` (proxy objects, slower), or an external store (Redis, a DB, Kafka).

T07-Q13 How do you handle timeouts and cancellation in asyncio?

Wrap awaits with `asyncio.timeout(seconds)` (3.11) or `asyncio.wait_for`. On timeout the inner task is cancelled via `CancelledError`. Coroutines should let `CancelledError` propagate (clean up in `finally`) so shutdown works.

```
import asyncio
async def slow(): await asyncio.sleep(5)
async def main():
    try:
        async with asyncio.timeout(1):
            await slow()
    except TimeoutError:
        print("timed out")
asyncio.run(main())
```

T07-Q14 What is the producer-consumer pattern? Implement it with `asyncio.Queue`.

Producers put work on a queue and consumers take it off, decoupling rates and giving back-pressure with `maxsize`. The same shape appears in Kafka consumers feeding worker pools.

```

import asyncio
async def producer(q):
    for i in range(5):
        await q.put(i)
    await q.put(None)          # poison pill
async def consumer(q):
    while (item := await q.get()) is not None:
        print("processed", item)
        q.task_done()
async def main():
    q = asyncio.Queue(maxsize=2)
    await asyncio.gather(producer(q), consumer(q))
asyncio.run(main())

```

Worked example: Benchmark: same I/O task with sequential vs threads vs asyncio

```

import asyncio, time
from concurrent.futures import ThreadPoolExecutor

def io_task(_): time.sleep(0.2)
async def aio_task(_): await asyncio.sleep(0.2)

N = 20
t = time.time(); [io_task(i) for i in range(N)]
print("sequential", round(time.time() - t, 2))          # ~4.0s

t = time.time()
with ThreadPoolExecutor(20) as ex: list(ex.map(io_task, range(N)))
print("threads   ", round(time.time() - t, 2))          # ~0.2s

async def main(): await asyncio.gather(*(aio_task(i) for i in range(N)))
t = time.time(); asyncio.run(main())
print("asyncio   ", round(time.time() - t, 2))          # ~0.2s

```

T08 Error Handling

Interviewers want to see clean try/except/else/finally usage, custom exception hierarchies, exception chaining, logging, and resilience patterns (retry with backoff). Modern Python adds `ExceptionGroup` and `except*`.

Must-know points

- try -> except (handle) -> else (runs if no exception) -> finally (always runs).
- Catch specific exceptions; never a bare `except:` (it also catches `KeyboardInterrupt/SystemExit`).
- `raise NewError(...) from err` keeps the root cause (`__cause__`).
- Build a custom hierarchy rooted in one app base exception.
- Log with `logger.exception()` inside except to capture the traceback.

Interview questions & answers

T08-Q01 Explain try / except / else / finally with an example.

`try` holds code that may fail. `except` handles specific errors. `else` runs only if no exception occurred — put the 'success path' there so its errors are not accidentally caught by the `except`. `finally` always runs (cleanup), even after return or an unhandled exception.

```
def read_config(path):
    try:
        f = open(path)
    except FileNotFoundError:
        return {}
    else:
        with f:
            return f.read()
    finally:
        print("read_config finished")
```

T08-Q02 Why is a bare `except:` or `except Exception:` pass bad?

A bare `except` catches everything including `KeyboardInterrupt`, `SystemExit` and `GeneratorExit`, making programs impossible to stop cleanly. Swallowing exceptions silently hides bugs. Catch the narrowest exception you can handle, and if you must catch broadly, log it and usually re-raise.

T08-Q03 Describe Python's exception hierarchy.

`BaseException` is the root. Under it: `SystemExit`, `KeyboardInterrupt`, `GeneratorExit`, `BaseExceptionGroup`, and `Exception`. Almost everything you catch derives from `Exception`: `ValueError`, `TypeError`, `KeyError/IndexError` (via `LookupError`), `OSError` (`FileNotFoundError`, `ConnectionError`, `TimeoutError`), `ArithmeticError` (`ZeroDivisionError`), `RuntimeError`, `StopIteration`, etc.

T08-Q04 How do you create custom exceptions? Show a good hierarchy.

Subclass `Exception` (not `BaseException`). Create one base for your application so callers can catch all of your errors, then specific subclasses carrying useful context.

```

class AppError(Exception):
    '''Base for all app errors.'''
class ValidationError(AppError):
    def __init__(self, field, msg):
        super().__init__(f"{field}: {msg}")
        self.field = field
class PaymentError(AppError): pass
class RetryableError(AppError): pass      # transient: safe to retry

try:
    raise ValidationError("amount", "must be positive")
except AppError as e:
    print(type(e).__name__, e, e.field)

```

T08-Q05 What is exception chaining? `raise ... from ...`?

When you translate a low-level error into a domain error, use `raise NewError(...) from err`. The original is stored in `__cause__` and both tracebacks are printed ('The above exception was the direct cause...'). Implicit chaining (`__context__`) happens automatically if you raise inside an except without `from`. `from None` suppresses the context.

```

import json
class ConfigError(Exception): pass
def load(s):
    try:
        return json.loads(s)
    except json.JSONDecodeError as e:
        raise ConfigError("invalid config file") from e

```

T08-Q06 How do you re-raise an exception while preserving the traceback?

Use a bare `raise` inside the except block. `raise e` also works in Python 3 (traceback is kept on the object) but adds a line; bare `raise` is the idiom.

```

import logging
log = logging.getLogger(__name__)
def process(x):
    try:
        return 1 / x
    except ZeroDivisionError:
        log.exception("bad input %s", x)  # logs with traceback
        raise                          # re-raise original

```

T08-Q07 EAFP vs LBYL — which is Pythonic?

EAFP (Easier to Ask Forgiveness than Permission): try the operation and handle the exception. LBYL (Look Before You Leap): check conditions first. Python generally favours EAFP because it avoids race conditions (e.g. a file deleted between `os.path.exists` and `open`) and double lookups. Use LBYL when failure is common and exceptions would be expensive.

```

# LBYL
if "key" in d: v = d["key"]
# EAFP
try: v = d["key"]
except KeyError: v = None

```

T08-Q08 What are ExceptionGroup and except* (Python 3.11)?

An `ExceptionGroup` bundles several unrelated exceptions raised together, e.g. by concurrent tasks in `asyncio.TaskGroup`. `except*` lets you handle each type within the group separately; unhandled ones propagate.

```
try:
    raise ExceptionGroup("batch", [ValueError("bad row"), KeyError("id")])
except* ValueError as eg:
    print("value errors:", eg.exceptions)
except* KeyError as eg:
    print("key errors:", eg.exceptions)
```

T08-Q09 What happens when `return` appears in both `try` and `finally`?

The `finally` return wins and silently discards both the `try`'s return value and any in-flight exception. It is a known footgun (Python 3.14 warns about it). Never return from `finally`.

```
def f():
    try:
        return "try"
    finally:
        return "finally"
print(f()) # 'finally'
```

T08-Q10 How do you add context to an exception without changing its type? (3.11)

Use `err.add_note('...')`. Notes are printed below the traceback. Useful in loops to record which record failed.

```
rows = ["1", "x"]
for i, r in enumerate(rows):
    try:
        int(r)
    except ValueError as e:
        e.add_note(f"row index {i}")
        raise
```

T08-Q11 How do you handle errors in a thread or process pool?

An exception inside a worker is captured in the `Future` and re-raised when you call `future.result()`. Always check results (or use `as_completed` with `try/except` per future), otherwise failures are silent. For threads started manually, override `threading.excepthook`.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
def job(x):
    if x == 2: raise ValueError("bad")
    return x
with ThreadPoolExecutor() as ex:
    futs = {ex.submit(job, i): i for i in range(4)}
    for f in as_completed(futs):
        try: print(f.result())
        except ValueError as e: print("job", futs[f], "failed:", e)
```

T08-Q12 Design a retry strategy for transient failures.

Retry only transient errors (timeouts, 5xx, connection resets) — never validation errors. Use exponential backoff with jitter to avoid thundering herds, cap the attempts and total time, make the operation idempotent, and log each attempt. Libraries: `tenacity`, `backoff`. After final failure, surface the error or send to a DLQ.


```
import random, time
def retry(fn, attempts=5, base=0.2, cap=5.0, retry_on=(TimeoutError, ConnectionError)):
    for i in range(attempts):
        try:
            return fn()
        except retry_on:
            if i == attempts - 1: raise
            sleep = min(cap, base * 2 ** i) * random.uniform(0.5, 1.5)
            time.sleep(sleep)
```

T08-Q13 How should a REST API map exceptions to responses?

Raise domain exceptions in the service layer and translate them centrally (exception handlers/middleware) into HTTP codes: validation -> 400/422, not found -> 404, conflict -> 409, auth -> 401/403, unexpected -> 500 with a correlation ID and no internal details leaked. Log the full traceback server-side.

T08-Q14 What is the cost of exceptions in Python?

Entering a try block is almost free (zero-cost in 3.11+). Raising and catching is relatively expensive (creates an object and traceback) — fine for exceptional paths, but avoid using exceptions for very frequent normal control flow inside hot loops.

Worked example: FastAPI-style central error mapping with custom exceptions

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
import logging, uuid

class AppError(Exception):
    status = 500; code = "INTERNAL"
class NotFound(AppError):
    status = 404; code = "NOT_FOUND"
class Conflict(AppError):
    status = 409; code = "CONFLICT"

app = FastAPI()
log = logging.getLogger("api")

@app.exception_handler(AppError)
async def handle_app_error(request: Request, exc: AppError):
    cid = str(uuid.uuid4())
    log.error("cid=%s %s", cid, exc, exc_info=exc)
    return JSONResponse(status_code=exc.status,
        content={"code": exc.code, "message": str(exc), "correlation_id": cid})

@app.get("/orders/{oid}")
async def get_order(oid: int):
    raise NotFound(f"order {oid} not found")
```

T09 Pandas for Data Handling

Expect practical questions: loading, selecting (loc/iloc), missing data, groupby/agg, merge/join, pivot, apply vs vectorisation, performance with large files, and dtypes. Many interviews include a live 'transform this DataFrame' task.

Must-know points

- Series = labelled 1-D array; DataFrame = table of Series sharing an index.
- **loc** = label based (end inclusive); **iloc** = position based (end exclusive).
- Vectorised ops >> apply >> iterrows for speed.
- groupby follows split-apply-combine; use named aggregation.
- For large data: chunksize, usecols, category dtype, pyarrow backend, Parquet.

Interview questions & answers

T09-Q01 What is the difference between a Series and a DataFrame?

A Series is a one-dimensional labelled array with a single dtype. A DataFrame is a two-dimensional table: a dict-like collection of Series (columns) aligned on a shared row index, where each column can have a different dtype.

```
import pandas as pd
s = pd.Series([10, 20], index=["a", "b"])
df = pd.DataFrame({"name": ["Ravi", "Asha"], "salary": [100, 120]})
print(df.dtypes)
```

T09-Q02 Difference between loc and iloc?

loc selects by label (index/column names), supports boolean masks, and slicing is end-inclusive. **iloc** selects by integer position and slicing is end-exclusive like Python lists.

```
import pandas as pd
df = pd.DataFrame({"x": [1, 2, 3]}, index=["a", "b", "c"])
print(df.loc["a":"b"])      # rows a and b (inclusive)
print(df.iloc[0:2])         # rows 0 and 1
print(df.loc[df.x > 1, "x"])
```

T09-Q03 How do you handle missing data?

Detect with **isna()/isna().sum()**. Remove with **dropna(subset=..., thresh=...)**. Fill with **fillna(value)**, column-wise dicts, statistics (**fillna(df.col.median())**), forward/back fill (**ffill**, **bfill**) for time series, or **interpolate()**. Choose based on business meaning — never blindly fill with 0.

```
import pandas as pd
df = pd.DataFrame({"city": ["A", None, "B"], "sales": [100, None, 50]})
print(df.isna().sum())
df["sales"] = df["sales"].fillna(df["sales"].median())
df["city"] = df["city"].fillna("UNKNOWN")
```

T09-Q04 Explain groupby with multiple aggregations.

groupby splits rows by key, applies aggregation functions to each group, and combines results. Named aggregation gives clear output column names.

```
import pandas as pd
df = pd.DataFrame({"dept": ["IT", "IT", "HR"], "emp": ["a", "b", "c"],
                  "salary": [100, 150, 90]})
out = df.groupby("dept").agg(
    headcount=("emp", "count"),
    avg_salary=("salary", "mean"),
    max_salary=("salary", "max"),
).reset_index()
print(out)
```

T09-Q05 What is the difference between merge, join and concat?

merge is SQL-style joining on columns or index (`how='inner'|'left'|'right'|'outer'|'cross'`). **join** is a convenience for joining on the index. **concat** stacks DataFrames vertically (`axis=0`) or side by side (`axis=1`) without key matching. Use `validate='one_to_one'` and `indicator=True` in merge to catch bad joins.

```
import pandas as pd
orders = pd.DataFrame({"oid": [1, 2, 3], "cid": [10, 20, 30]})
cust = pd.DataFrame({"cid": [10, 20], "name": ["Ravi", "Asha"]})
m = orders.merge(cust, on="cid", how="left", indicator=True)
print(m)  # oid 3 has NaN name and _merge='left_only'
```

T09-Q06 apply vs map vs vectorisation — which is fastest?

Vectorised operations (column arithmetic, `.str`, `.dt`, `np.where`, `np.select`) run in C and are fastest. `Series.map` is good for dict lookups. `apply` runs Python per row/element and is slow; `iterrows` is slowest. Rewrite row-wise apply into vectorised expressions where possible.

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"amt": np.random.randint(0, 1000, 1_000_000)})
# slow: df["tier"] = df["amt"].apply(lambda a: "high" if a > 500 else "low")
df["tier"] = np.where(df["amt"] > 500, "high", "low")  # fast
df["band"] = np.select([df.amt < 100, df.amt < 500], ["S", "M"], "L")
```

T09-Q07 How do you process a CSV that is larger than memory?

Read in chunks with `chunksize` and aggregate incrementally; load only needed columns (`usecols`); set compact dtypes (`dtype, category`); use `engine='pyarrow'`; convert to Parquet for repeated use. For bigger jobs consider Polars, DuckDB, Dask or Spark.

```
import pandas as pd
totals = {}
for chunk in pd.read_csv("sales.csv", usecols=["region", "amount"],
                        dtype={"region": "category", "amount": "float"}, chunksize=500_000):
    for region, amt in chunk.groupby("region", observed=True)["amount"].sum().items():
        totals[region] = totals.get(region, 0) + amt
```

T09-Q08 How do you reduce DataFrame memory usage?

Check with `df.memory_usage(deep=True)`. Downcast numerics (`pd.to_numeric(..., downcast='integer')`), convert low-cardinality strings to `category`, use nullable/arrow dtypes (`dtype_backend='pyarrow'`), parse dates properly, and drop unneeded columns early.

T09-Q09 What is the SettingWithCopyWarning and how do you avoid it?

It appears when you modify what may be a copy of a slice (chained indexing like `df[df.a > 0]['b'] = 1`), so the change might not reach the original. Use a single `.loc[mask, 'b'] = 1`, or take an explicit `.copy()` when you really want a new frame. Pandas 3.0 makes Copy-on-Write the default, which removes the ambiguity.

T09-Q10 Explain pivot_table vs pivot vs melt.

`pivot` reshapes long -> wide and fails on duplicate index/column pairs. `pivot_table` does the same but aggregates duplicates (`aggfunc`) and supports margins. `melt` is the inverse: wide -> long.

```
import pandas as pd
df = pd.DataFrame({"month": ["Jan", "Jan", "Feb"], "region": ["N", "S", "N"],
                  "sales": [100, 80, 120]})
wide = df.pivot_table(index="month", columns="region", values="sales",
                      aggfunc="sum", fill_value=0)
long = wide.reset_index().melt(id_vars="month", value_name="sales")
```

T09-Q11 How do you work with dates and time series in pandas?

Parse with `pd.to_datetime` (or `parse_dates`), use the `.dt` accessor for parts, set a `DatetimeIndex` and `resample('D' | 'W' | 'M')` for aggregation, `rolling(window)` for moving averages, and `shift/diff/pct_change` for period-over-period metrics.

```
import pandas as pd
df = pd.DataFrame({"ts": pd.date_range("2026-09-01", periods=10, freq="D"),
                  "sales": range(10)})
df = df.set_index("ts")
print(df.resample("W").sum())
df["ma3"] = df["sales"].rolling(3).mean()
```

T09-Q12 How do you find and remove duplicates?

`df.duplicated(subset=[...], keep='first')` returns a boolean mask; `drop_duplicates(subset, keep='last')` removes them. For 'keep the latest record per key', sort by timestamp first then drop duplicates on the key.

```
import pandas as pd
events = pd.DataFrame({"id": [1, 1, 2], "ts": [1, 3, 2], "v": ["a", "b", "c"]})
latest = events.sort_values("ts").drop_duplicates("id", keep="last")
```

T09-Q13 What is the difference between transform and agg in groupby?

`agg` reduces each group to one row. `transform` returns a result with the same shape as the input, aligned to the original rows — perfect for adding group-level features such as 'percentage of department total'.

```
import pandas as pd
df = pd.DataFrame({"dept": ["IT", "IT", "HR"], "sal": [100, 300, 50]})
df["dept_total"] = df.groupby("dept")["sal"].transform("sum")
df["share"] = df["sal"] / df["dept_total"]
```

T09-Q14 How would you rank or get top-N per group?

Use `groupby().rank()`, or sort then `groupby().head(n)`, or `nlargest` inside `groupby`.

```
import pandas as pd
df = pd.DataFrame({"dept": ["IT", "IT", "IT", "HR"], "emp": list("abcd"),
                  "sal": [100, 300, 200, 50]})
top2 = df.sort_values("sal", ascending=False).groupby("dept").head(2)
df["rank"] = df.groupby("dept")["sal"].rank(ascending=False, method="dense")
```

Worked example: End-to-end: clean, enrich and summarise Kafka-exported order events

```
import pandas as pd

raw = pd.DataFrame({
    "order_id": [1, 2, 2, 3, 4],
    "ts": ["2026-09-01 10:00", "2026-09-01 11:00", "2026-09-01 11:05",
          "2026-09-02 09:30", None],
    "customer": ["c1", "c2", "c2", "c1", "c3"],
    "amount": ["100", "250", "250", None, "80"],
    "status": ["PAID", "PAID", "PAID", "FAILED", "PAID"],
})

df = (raw
      .dropna(subset=["ts"])
      .assign(ts=lambda d: pd.to_datetime(d.ts),
              amount=lambda d: pd.to_numeric(d.amount).fillna(0))
      .sort_values("ts")
      .drop_duplicates("order_id", keep="last"))    # idempotent dedupe

daily = (df[df.status == "PAID"]
        .groupby([df.ts.dt.date.rename("day"), "customer"])
        .agg(orders=("order_id", "nunique"), revenue=("amount", "sum"))
        .reset_index())

print(daily)
```

T10 Web Frameworks: Django / Flask / FastAPI (FastAPI in depth)

Know when to pick each framework, then go deep on one. FastAPI is the recommended in-depth choice for a Python backend + GenAI/Kafka profile: async, Pydantic validation, dependency injection, automatic OpenAPI docs. Be ready to compare it to Spring Boot too.

Must-know points

- Django: batteries-included (ORM, admin, auth, migrations), sync-first, WSGI/ASGI.
- Flask: micro-framework, WSGI, you choose the extensions (SQLAlchemy, Marshmallow).
- FastAPI: ASGI on Starlette, Pydantic v2 validation, type-hint driven, async-first.
- Key FastAPI concepts: path/query/body params, Depends(), response_model, middleware, BackgroundTasks, lifespan.
- Production: Uvicorn/Gunicorn workers, async DB drivers, structured logging, tests with TestClient.

Interview questions & answers

T10-Q01 Compare Django, Flask and FastAPI. When would you pick each?

Django suits full-featured, data-driven web apps where you want ORM, admin, auth and migrations out of the box (CMS, internal portals). Flask suits small services or when you want full control over components. FastAPI suits API-first microservices needing high I/O concurrency, strong request/response validation and automatic OpenAPI docs (ML/GenAI inference APIs, event-driven services). For a microservice ecosystem with Kafka and LLM calls, FastAPI is usually the best fit.

T10-Q02 What are WSGI and ASGI?

WSGI is the synchronous interface between Python web apps and servers (Gunicorn, uWSGI): one request per worker thread at a time. ASGI is its async successor that supports async/await, WebSockets, HTTP/2 and long-lived connections; servers are Uvicorn, Hypercorn, Daphne. Flask is WSGI; FastAPI/Starlette are ASGI; Django supports both.

T10-Q03 How does FastAPI use type hints and Pydantic?

FastAPI reads function signatures: path parameters, query parameters (simple types), and request bodies (Pydantic models). Pydantic validates and converts incoming data, returning a 422 with details on failure; `response_model` filters and validates output. The same hints generate the OpenAPI schema and Swagger UI at /docs.

```

from fastapi import FastAPI
from pydantic import BaseModel, Field

class OrderIn(BaseModel):
    customer_id: int
    amount: float = Field(gt=0)
    currency: str = "INR"

class OrderOut(BaseModel):
    id: int
    status: str

app = FastAPI()

@app.post("/orders", response_model=OrderOut, status_code=201)
async def create_order(order: OrderIn, notify: bool = False):
    return OrderOut(id=1, status="CREATED")

```

T10-Q04 Explain dependency injection in FastAPI with Depends().

Depends(callable) tells FastAPI to call the dependency for each request and inject the result. Dependencies can depend on others, are cached per request, and can be **yield** dependencies for setup/teardown (e.g. DB session). You can override them in tests with **app.dependency_overrides**.

```

from fastapi import Depends, Header, HTTPException

async def get_db():
    db = SessionLocal()
    try:
        yield db          # teardown after response
    finally:
        db.close()

async def current_user(authorization: str = Header(...)):
    if not authorization.startswith("Bearer "):
        raise HTTPException(401, "invalid token")
    return {"id": 7, "roles": ["admin"]}

@app.get("/me")
async def me(user=Depends(current_user), db=Depends(get_db)):
    return user

```

T10-Q05 When should an endpoint be **async def** vs **plain def** in FastAPI?

Use **async def** when everything inside is non-blocking (awaiting async DB drivers, `httpx.AsyncClient`, `aiokafka`). Use **plain def** when calling blocking libraries (requests, sync SQLAlchemy, boto3): FastAPI runs sync endpoints in a thread pool so they don't block the event loop. The worst case is **async def** that calls blocking code — it freezes all requests on that worker.

T10-Q06 How do you structure a larger FastAPI application?

Split into routers (**APIRouter**) per domain with prefixes and tags; layers for schemas (Pydantic), models (ORM), repositories, services (business logic), and dependencies; settings via **pydantic-settings**; a **lifespan** handler for startup/shutdown resources (DB pool, Kafka producer). Include routers in **main.py**.

```

from contextlib import asynccontextmanager
from fastapi import FastAPI, APIRouter

router = APIRouter(prefix="/orders", tags=["orders"])
@router.get("/{oid}")
async def get(oid: int): return {"id": oid}

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.producer = await start_kafka_producer() # startup
    yield
    await app.state.producer.stop() # shutdown

app = FastAPI(lifespan=lifespan)
app.include_router(router)

```

T10-Q07 How do you add middleware in FastAPI? Give a use case.

Middleware wraps every request/response: logging, correlation IDs, timing, CORS, GZip. Use `@app.middleware('http')` or add Starlette middleware classes.

```

import time, uuid
@app.middleware("http")
async def add_timing(request, call_next):
    cid = request.headers.get("X-Correlation-ID", str(uuid.uuid4()))
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Correlation-ID"] = cid
    response.headers["X-Process-Time"] = f"{time.perf_counter()-start:.4f}"
    return response

```

T10-Q08 How do you implement authentication (JWT/OAuth2) in FastAPI?

Use `OAuth2PasswordBearer` to extract the bearer token, decode/verify the JWT (PyJWT/python-jose) in a dependency, and raise 401 on failure. Protect routes by depending on `get_current_user`; implement role checks as further dependencies. For enterprise SSO, validate tokens from an IdP (Cognito, Keycloak, Azure AD) using its JWKS.

T10-Q09 What are BackgroundTasks and when are they not enough?

`BackgroundTasks` runs a function after the response is sent (send an email, write an audit log) in the same process. It is not durable: if the process crashes the task is lost, and there are no retries. For reliable or heavy work use a queue (Celery, RQ, Arq) or publish to Kafka and process in a consumer.

T10-Q10 How do you test a FastAPI application?

Use `TestClient` (sync, built on httpx) or `httpx.AsyncClient` with `ASGITransport` for async tests, with pytest. Override dependencies to inject fakes (test DB, mock user).

```

from fastapi.testclient import TestClient
def fake_user(): return {"id": 1, "roles": ["admin"]}
app.dependency_overrides[current_user] = fake_user
client = TestClient(app)
def test_me():
    r = client.get("/me")
    assert r.status_code == 200 and r.json()["id"] == 1

```


T10-Q11 How do you run FastAPI in production?

Run Uvicorn workers under Gunicorn (`gunicorn -k uvicorn.workers.UvicornWorker -w 4`) or `uvicorn --workers`, behind a reverse proxy/load balancer; containerise with Docker, deploy on ECS/EKS; set worker count roughly to CPU cores; use connection pooling, health/readiness endpoints, structured JSON logs, metrics (Prometheus), tracing (OpenTelemetry) and graceful shutdown via lifespan.

T10-Q12 Explain the Django request lifecycle and its ORM basics.

Request -> WSGI/ASGI handler -> middleware chain -> URL resolver (urls.py) -> view (function or class-based) -> ORM/templates -> response back through middleware. The ORM maps Model classes to tables; QuerySets are lazy and chainable; migrations track schema changes. Avoid N+1 queries with `select_related` (FK joins) and `prefetch_related` (separate query for M2M/reverse FK).

T10-Q13 Flask: explain application and request context and blueprints.

Flask uses context locals: `current_app/g` (application context) and `request/session` (request context) are available inside a request without passing them explicitly. Blueprints group routes, templates and static files into modular components registered on the app — similar to FastAPI routers.

T10-Q14 How does FastAPI compare with Spring Boot for a Java developer?

Controllers ~ routers/path operations; `@RequestBody`/DTO validation ~ Pydantic models; Spring DI ~ `Depends()`; filters/interceptors ~ middleware; `@ControllerAdvice` ~ `exception_handler`; Actuator ~ custom health endpoints/Prometheus instrumentator; Spring Kafka ~ confluent-kafka/aiokafka in a lifespan-managed client. FastAPI is lighter and async-native, but has fewer built-in enterprise modules.

Worked example: FastAPI service that validates an order and publishes it to Kafka

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from confluent_kafka import Producer, KafkaException
import asyncio, threading

class Order(BaseModel):
    order_id: str
    customer_id: int
    amount: float = Field(gt=0)

@asynccontextmanager
async def lifespan(app: FastAPI):
    p = Producer({"bootstrap.servers": "localhost:9092",
                 "enable.idempotence": True, "acks": "all"})
    stop = threading.Event()
    def poll_loop():
        # delivery callbacks fire inside poll()
        while not stop.is_set():
            p.poll(0.1)
    t = threading.Thread(target=poll_loop, daemon=True); t.start()
    app.state.producer = p
    yield
    stop.set(); t.join(); p.flush(10) # graceful shutdown

app = FastAPI(lifespan=lifespan)

@app.post("/orders", status_code=202)
async def create(order: Order):
    loop = asyncio.get_running_loop()
    fut = loop.create_future()
    def on_delivery(err, msg):
        # runs on the poll thread
        if err:
            loop.call_soon_threadsafe(fut.set_exception, KafkaException(err))
        else:
            loop.call_soon_threadsafe(fut.set_result, msg.offset())
    app.state.producer.produce("orders", key=order.order_id,
                               value=order.model_dump_json(), on_delivery=on_delivery)
    try:
        offset = await asyncio.wait_for(fut, timeout=10)
    except Exception as e:
        raise HTTPException(503, f"publish failed: {e}")
    return {"status": "ACCEPTED", "offset": offset}
```

T11 Kafka Core: Topics, Partitions, Replication, Producers, Consumers

Foundational Kafka questions come up in every event-driven interview. Understand the log abstraction, how partitions give ordering and parallelism, how replication (leader, followers, ISR) gives durability, and how consumer groups share partitions.

Must-know points

- Topic = named, append-only, partitioned log; messages are retained by time/size, not deleted on read.
- Ordering is guaranteed only within a partition; key -> partition via $\text{hash}(\text{key}) \% \text{partitions}$.
- Replication factor N: one leader + N-1 followers; ISR = replicas caught up with the leader.
- Consumer group: each partition is read by exactly one consumer in the group.
- Offsets are committed per (group, topic, partition) in the internal `__consumer_offsets` topic.

Interview questions & answers

T11-Q01 What is Kafka and why is it used?

Apache Kafka is a distributed, durable, partitioned commit log used as an event streaming platform. It decouples producers and consumers, handles very high throughput via sequential disk I/O, batching and zero-copy, retains data for replay, and scales horizontally. Common uses: microservice event bus, CDC pipelines, log/metric aggregation, stream processing and feeding data lakes or ML/GenAI pipelines.

T11-Q02 Explain topic, partition, offset, broker and cluster.

A broker is a Kafka server; brokers form a cluster. A topic is a logical stream of records, split into partitions. Each partition is an ordered, immutable log stored on a broker; each record in it has a sequential offset. Partitions are the unit of parallelism and ordering. Modern Kafka (3.x/4.0) manages cluster metadata with KRaft controllers instead of ZooKeeper (ZooKeeper support was removed in 4.0).

T11-Q03 How does Kafka decide which partition a message goes to?

If a partition is specified explicitly, that one is used. Otherwise if the record has a key, the default partitioner hashes the key (murmur2 in the Java client; librdkafka's default for confluent-kafka-python is `consistent_random` based on CRC32 — set `partitioner='murmur2_random'` to match Java) and takes modulo the partition count, so the same key always lands on the same partition. With no key, the client uses a sticky partitioner that fills a batch for one partition before switching, improving batching.

```
from confluent_kafka import Producer
p = Producer({"bootstrap.servers": "localhost:9092",
             "partitioner": "murmur2_random"}) # Java-compatible hashing
p.produce("orders", key="customer-42", value=b"...") # same key -> same partition
```

T11-Q04 How does Kafka guarantee ordering?

Only within a single partition. To keep all events of an entity in order (e.g. an order's CREATED -> PAID -> SHIPPED), use the entity ID as the key. With retries enabled, keep ordering by enabling idempotence (which allows up to 5 in-flight requests safely) — otherwise set `max.in.flight.requests.per.connection=1`. Increasing the partition count later changes key-to-partition mapping and can break ordering for existing keys.

T11-Q05 How do you choose the number of partitions?

Partitions cap consumer parallelism (max active consumers in a group = partitions). Estimate: $\text{partitions} \geq \max(\text{target throughput} / \text{per-partition producer throughput}, \text{target throughput} / \text{per-consumer throughput})$, plus headroom for growth, since adding partitions later remaps keys. Too many partitions increase open files, metadata, rebalance and recovery time. Tens to low hundreds per topic is typical.

T11-Q06 Explain replication: leader, followers, ISR.

Each partition has a replication factor (commonly 3). One replica is the leader and handles all writes (and by default reads); followers fetch from the leader. The In-Sync Replica set contains replicas that are caught up within `replica.lag.time.max.ms`. If the leader dies, the controller elects a new leader from the ISR, so no committed data is lost.

T11-Q07 What do acks=0, acks=1 and acks=all mean?

acks=0: producer doesn't wait for any acknowledgement — fastest, may lose data. acks=1: leader writes to its log and acknowledges — data lost if the leader fails before followers copy it. acks=all (-1): leader waits until all ISR replicas have the record — strongest durability. Combine acks=all with `min.insync.replicas=2` and RF=3 so a write fails rather than being stored on only one broker.

T11-Q08 What is min.insync.replicas and what happens when ISR falls below it?

A topic/broker setting giving the minimum ISR size required for an acks=all write to succeed. With RF=3 and `min.insync.replicas=2` you tolerate one broker failure. If ISR drops below 2, producers with acks=all get `NotEnoughReplicas` errors (writes rejected) instead of silently reducing durability; reads still work.

T11-Q09 What is unclean leader election?

If all ISR replicas are gone, Kafka can elect an out-of-sync replica as leader (`unclean.leader.election.enable=true`) to restore availability, but messages it never received are lost. The default is false: prefer consistency over availability for important data.

T11-Q10 How do consumer groups work?

Consumers sharing a `group.id` form a group; the group coordinator assigns each partition to exactly one consumer in the group, so work is shared. Different groups each get a full copy of the stream (pub-sub). If there are more consumers than partitions, extras sit idle. Each group tracks its own committed offsets.

```
from confluent_kafka import Consumer
c = Consumer({"bootstrap.servers": "localhost:9092",
             "group.id": "billing-service",
             "auto.offset.reset": "earliest",
             "enable.auto.commit": False})
c.subscribe(["orders"])
while True:
    msg = c.poll(1.0)
    if msg is None: continue
    if msg.error(): print(msg.error()); continue
    print(msg.partition(), msg.offset(), msg.key(), msg.value())
    c.commit(message=msg, asynchronous=False)
```

T11-Q11 What is a rebalance, and how do you reduce its impact?

When consumers join/leave/crash or subscriptions change, partitions are reassigned. With the eager protocol all consumers stop and give up partitions (stop-the-world). Reduce impact with the cooperative-sticky assignor (incremental rebalancing), static membership (`group.instance.id`) for rolling restarts, tuning `session.timeout.ms/max.poll.interval.ms`, keeping processing within poll intervals, and committing offsets in the `on_revoke` callback. Kafka 4.0 also brings the new server-side consumer group protocol (KIP-848).

T11-Q12 What is the difference between `session.timeout.ms` and `max.poll.interval.ms`?

`session.timeout.ms`: if the broker doesn't get heartbeats (sent by a background thread) within this time, the consumer is considered dead. `max.poll.interval.ms`: maximum time between calls to `poll()`; if your processing is too slow, the consumer leaves the group and triggers a rebalance even though heartbeats are fine. Slow processing -> raise `max.poll.interval.ms` or reduce batch size (`max.poll.records` / fetch settings).

T11-Q13 What does `auto.offset.reset` do?

Applies only when the group has no committed offset for a partition (new group) or the committed offset no longer exists (deleted by retention). `earliest` starts from the oldest available message, `latest` from new messages only, `none/error` raises an error.

T11-Q14 How does Kafka retention and log compaction work?

Delete policy removes old segments after `retention.ms` or when `retention.bytes` is exceeded. Compaction (`cleanup.policy=compact`) keeps at least the latest record per key and removes older versions; a record with a null value (tombstone) deletes the key after `delete.retention.ms`. Compacted topics are used for changelogs, state stores, and `__consumer_offsets`.

T11-Q15 Why is Kafka so fast?

Sequential appends to disk and reliance on the OS page cache, batching and compression (lz4/zstd) of records, zero-copy transfer (sendfile) from page cache to socket, partitioned parallelism across brokers, and a simple consumer-driven pull model where the broker doesn't track per-message state.

Worked example: Create a topic programmatically with AdminClient

```
from confluent_kafka.admin import AdminClient, NewTopic

admin = AdminClient({"bootstrap.servers": "localhost:9092"})
topic = NewTopic("orders", num_partitions=6, replication_factor=3,
                 config={"min.insync.replicas": "2",
                        "retention.ms": str(7 * 24 * 3600 * 1000),
                        "cleanup.policy": "delete"})
for name, fut in admin.create_topics([topic]).items():
    try:
        fut.result()
        print("created", name)
    except Exception as e:
        print("failed", name, e)

md = admin.list_topics(timeout=10)
for p in md.topics["orders"].partitions.values():
    print(p.id, "leader", p.leader, "isr", p.isrs)
```

T12 Designing Kafka Pipelines in Python and Delivery Semantics

This is where you show engineering judgement: at-most-once vs at-least-once vs exactly-once, how commit order creates each, idempotent producers, transactions (consume-transform-produce), and the practical differences between confluent-kafka-python and kafka-python.

Must-know points

- At-most-once: commit before processing. At-least-once: process then commit. Exactly-once: idempotent producer + transactions + `read_committed`.
- confluent-kafka-python wraps librdkafka (C): fastest, full feature set, transactions, Schema Registry.
- kafka-python is pure Python: simple, no C dependency, slower, fewer features.
- Idempotent producer removes duplicates caused by producer retries (per partition, per session).
- End-to-end exactly-once also needs idempotent sinks for external systems.

Interview questions & answers

T12-Q01 Explain the three delivery semantics in Kafka.

At-most-once: messages may be lost but never redelivered (commit offset first, then process). At-least-once: never lost but may be duplicated (process first, commit after success) — the most common default. Exactly-once: each message affects the result once — Kafka provides it for Kafka-to-Kafka flows via idempotent producers and transactions; for external sinks you need idempotent writes or transactional outbox/offset storage.

T12-Q02 Show at-most-once and at-least-once consumers in Python.

The only difference is the order of commit and processing (with auto-commit disabled).

```
# AT-MOST-ONCE: commit first
msg = consumer.poll(1.0)
consumer.commit(message=msg, asynchronous=False)
process(msg)                    # crash here -> message lost

# AT-LEAST-ONCE: process first
msg = consumer.poll(1.0)
process(msg)                    # crash here -> re-delivered later
consumer.commit(message=msg, asynchronous=False)
```

T12-Q03 Why is `enable.auto.commit=True` risky?

The client commits offsets periodically (every `auto.commit.interval.ms`) for messages returned by poll, regardless of whether your code finished processing them. A crash after the auto-commit but before processing loses messages; a crash after processing but before the next auto-commit causes duplicates. For reliable pipelines disable it and commit explicitly after processing. (In librdkafka a common pattern is `enable.auto.commit=true` with `enable.auto.offset.store=false` and calling `store_offsets()` after processing.)

T12-Q04 What is an idempotent producer and how do you enable it?

Set `enable.idempotence=True` (default on in modern clients). The broker assigns a producer ID and tracks per-partition sequence numbers, discarding duplicate batches caused by retries and preserving order with up to 5 in-flight requests. It forces `acks=all`. It covers one producer session — it does not deduplicate if your application sends the same business event twice.

```
from confluent_kafka import Producer
p = Producer({
    "bootstrap.servers": "localhost:9092",
    "enable.idempotence": True,      # implies acks=all, retries, in-flight<=5
    "compression.type": "zstd",
    "linger.ms": 10, "batch.size": 131072,
})
```

T12-Q05 How do Kafka transactions give exactly-once processing?

A transactional producer (with `transactional.id`) can atomically write output records to multiple partitions AND commit the consumer's offsets in the same transaction (`send_offsets_to_transaction`). Consumers reading the output use `isolation.level=read_committed`, so they never see aborted records. If the app crashes, the transaction aborts and the input is re-read from the last committed offset; the `transactional.id` fences zombie instances.

```
from confluent_kafka import Producer, Consumer
c = Consumer({"bootstrap.servers": "kafka:9092", "group.id": "enricher",
             "enable.auto.commit": False, "isolation.level": "read_committed"})
p = Producer({"bootstrap.servers": "kafka:9092",
             "transactional.id": "enricher-1"})
p.init_transactions()
c.subscribe(["orders"])
while True:
    msgs = c.consume(num_messages=100, timeout=1.0)
    if not msgs: continue
    p.begin_transaction()
    try:
        for m in msgs:
            if m.error(): continue
            p.produce("orders-enriched", key=m.key(), value=enrich(m.value()))
            p.send_offsets_to_transaction(c.position(c.assignment()),
                                         c.consumer_group_metadata())
        p.commit_transaction()
    except Exception:
        p.abort_transaction()
        # rewind to last committed offsets so the batch is reprocessed
        for tp in c.committed(c.assignment()): c.seek(tp)
```

T12-Q06 confluent-kafka-python vs kafka-python — which would you choose?

`confluent-kafka-python` wraps `librdkafka` (C), is maintained by Confluent, is much faster, and supports transactions, idempotence, Schema Registry serializers (Avro/Protobuf/JSON Schema), OAuth/SASL, admin API and AsyncIO producer/consumer (newer versions). `kafka-python` is pure Python and easy to install but slower and has had long maintenance gaps (a community fork, `kafka-python-ng`, and renewed releases exist). For production and exactly-once: `confluent-kafka`. `aiokafka` is an option for asyncio-native apps.

T12-Q07 Show the equivalent producer/consumer in kafka-python.

The API is closer to the Java client: `send()` returns a future, and serializers are functions.

```
from kafka import KafkaProducer, KafkaConsumer
import json
producer = KafkaProducer(bootstrap_servers="localhost:9092", acks="all",
    key_serializer=str.encode,
    value_serializer=lambda v: json.dumps(v).encode())
meta = producer.send("orders", key="o-1", value={"amt": 10}).get(timeout=10)
print(meta.partition, meta.offset)

consumer = KafkaConsumer("orders", bootstrap_servers="localhost:9092",
    group_id="billing", enable_auto_commit=False,
    auto_offset_reset="earliest",
    value_deserializer=lambda b: json.loads(b))
for msg in consumer:
    handle(msg.value)
    consumer.commit()
```

T12-Q08 How do producer delivery reports work in confluent-kafka-python?

`produce()` is asynchronous: it enqueues the message in a local buffer and returns immediately. Delivery results arrive through the `on_delivery` callback, which is invoked only when you call `poll()` or `flush()`. Call `poll(0)` regularly in the produce loop and `flush()` before shutdown; handle `BufferError` (local queue full) by polling and retrying.

```
def report(err, msg):
    if err: log.error("delivery failed %s", err)
    else:   log.debug("%s [%d] @ %d", msg.topic(), msg.partition(), msg.offset())

for rec in records:
    while True:
        try:
            p.produce("orders", key=rec["id"], value=json.dumps(rec), on_delivery=report)
            break
        except BufferError:
            p.poll(0.5)          # wait for queue space
    p.poll(0)                  # serve callbacks
p.flush(30)
```

T12-Q09 Synchronous vs asynchronous commits — trade-offs?

Synchronous commit (`asynchronous=False`) blocks until the broker confirms: safer, slower. Async commit is faster but a failure is only reported via callback and a later commit may overtake an earlier one. Common pattern: async commits during normal processing and a final synchronous commit on shutdown/partition revocation. Committing per batch instead of per message greatly improves throughput.

T12-Q10 How do you achieve exactly-once when the sink is a database?

Kafka transactions don't cover external systems. Options: (1) idempotent writes — upsert keyed by a business/event ID or (topic, partition, offset), so replays are harmless; (2) store the consumed offset in the same DB transaction as the results and, on start/assignment, seek to the stored offset; (3) transactional outbox for DB->Kafka direction. In practice: at-least-once delivery + idempotent consumer = effectively-once.

T12-Q11 How do you design a high-throughput producer?

Batch with `linger.ms` (5-50 ms) and larger `batch.size`, enable compression (zstd/lz4), keep idempotence on, use asynchronous produce with delivery callbacks rather than waiting per message, reuse one producer per process (it's thread-safe), tune `queue.buffering.max.messages/kbytes`, and choose keys that spread load evenly (avoid hot partitions).

T12-Q12 How do you scale a slow consumer?

Add consumers up to the partition count; increase partitions if needed; process in batches (`consume(num_messages=N)`); move CPU-heavy work to a process pool or async I/O while keeping per-key ordering (e.g. route by key to a fixed worker); commit only contiguous processed offsets; and tune fetch sizes. Watch consumer lag (kafka-consumer-groups, Burrow, Prometheus exporter) as the key metric.

T12-Q13 What is the transactional outbox pattern?

A service writes its business change and an 'outbox' event row in the same local DB transaction. A separate relay (poller or CDC with Debezium) publishes outbox rows to Kafka. This avoids the dual-write problem where the DB commit succeeds but the Kafka publish fails (or vice versa).

T12-Q14 How would you design an end-to-end order-processing pipeline?

API (FastAPI) validates orders and writes to DB + outbox -> Debezium publishes to `orders` (key=order_id, RF=3, min ISR=2). Consumers: payment-service (at-least-once + idempotent upsert), enrichment stream (transactional consume-transform-produce into `orders-enriched`), analytics sink to S3/warehouse. Failures go to retry topics and a DLQ. Schemas in Schema Registry with BACKWARD compatibility. Monitor lag, error rate, DLQ depth; alert on growth.

Worked example: Reusable at-least-once consumer with batch commit and graceful shutdown

```
import signal, json, logging
from confluent_kafka import Consumer, KafkaException

log = logging.getLogger("consumer")
running = True
def stop(*_):
    global running; running = False
    signal.signal(signal.SIGTERM, stop); signal.signal(signal.SIGINT, stop)

def on_revoke(consumer, partitions):
    try: consumer.commit(asynchronous=False)      # commit before losing them
    except KafkaException: pass

c = Consumer({"bootstrap.servers": "localhost:9092", "group.id": "billing",
             "enable.auto.commit": False, "auto.offset.reset": "earliest",
             "partition.assignment.strategy": "cooperative-sticky"})
c.subscribe(["orders"], on_revoke=on_revoke)
try:
    while running:
        batch = c.consume(num_messages=200, timeout=1.0)
        for m in batch:
            if m.error():
                log.error(m.error()); continue
            handle(json.loads(m.value()))        # must be idempotent
        if batch:
            c.commit(asynchronous=False)         # at-least-once
finally:
    c.close()                                    # leaves group cleanly
```

T13 Kafka Failure Handling: Retry Topics, DLQ, Consumer Idempotency

Real systems fail: downstream APIs time out, messages are malformed, consumers crash mid-batch. Interviewers look for a layered strategy — classify errors, retry transient ones without blocking the partition, park poison messages in a DLQ, and make consumers idempotent so redelivery is harmless.

Must-know points

- Classify: transient (retry) vs permanent/poison (DLQ immediately).
- In-place retries block the partition; retry topics with delays keep the main flow moving.
- DLQ message = original payload + headers (error, stack, source topic/partition/offset, attempts).
- Idempotent consumer: dedupe on event ID / upsert / processed-events table.
- Retry topics trade away strict per-key ordering — call that out.

Interview questions & answers

T13-Q01 What kinds of failures occur in a Kafka consumer?

Transient: downstream timeouts, 5xx, DB deadlocks, network blips, rate limiting. Permanent/poison pill: malformed JSON, schema mismatch, failed business validation, missing referenced data that will never exist. Infrastructure: consumer crash, rebalance mid-processing, broker failover. Each needs a different response: retry, DLQ, or rely on offset replay + idempotency.

T13-Q02 What is a poison pill message and how do you handle it?

A message that always fails processing (e.g. undeserialisable bytes). If you retry forever, the consumer is stuck at that offset and lag grows for the whole partition. Catch deserialisation/validation errors, send the raw message with error metadata to a DLQ, commit its offset and move on. Alert on DLQ activity.

T13-Q03 Why not just retry in a loop inside the consumer?

Short in-memory retries (a few attempts with small backoff) are fine for brief blips. Long retries block the partition (head-of-line blocking), delay every other message, and can exceed `max.poll.interval.ms` causing a rebalance storm. For longer delays, hand the message off to a retry topic and continue.

T13-Q04 Explain the retry-topic pattern.

On a transient failure, publish the message to `orders.retry.1` (e.g. 1 minute delay) with an attempt counter header and commit the original. A retry consumer waits until `timestamp + delay` before reprocessing; on failure it forwards to `orders.retry.2` (e.g. 10 minutes), and so on. After the last tier, it goes to `orders.dlq`. Spring Kafka's `@RetryableTopic` implements this non-blocking pattern; in Python you build it yourself.

```
RETRY_TIERS = [("orders.retry.1m", 60), ("orders.retry.10m", 600)]
DLQ = "orders.dlq"

def route_failure(producer, msg, err, attempt):
    headers = dict(msg.headers() or [])
    headers.update({"x-error": str(err).encode(),
                    "x-attempt": str(attempt + 1).encode(),
                    "x-origin-topic": (headers.get("x-origin-topic") or msg.topic()).encode(),
                    "x-origin-offset": str(msg.offset()).encode()})
    target = RETRY_TIERS[attempt][0] if attempt < len(RETRY_TIERS) else DLQ
    producer.produce(target, key=msg.key(), value=msg.value(),
                     headers=list(headers.items()))
```

T13-Q05 How does a delayed retry consumer wait without breaking the group?

Check the message timestamp; if not yet due, `pause()` the partition, `seek()` back to the message's offset, and `resume()` later — keep calling `poll()` so the consumer stays in the group. Do not sleep for minutes inside the poll loop.

```
import time
def handle_retry(c, msg, delay_s):
    _, ts = msg.timestamp()
    due = ts / 1000 + delay_s
    if time.time() < due:
        tp = TopicPartition(msg.topic(), msg.partition(), msg.offset())
        c.pause([tp]); c.seek(tp)
        schedule_resume(tp, at=due)    # resume() when due; keep polling
        return False
    return True
```

T13-Q06 What should a DLQ message contain?

The original key, value and headers untouched, plus metadata headers: original topic/partition/offset, timestamp, consumer group, exception class and message, stack trace (truncated), attempt count, service version and failure time. This lets you debug and replay precisely. Keep DLQ retention long (weeks) and restrict who can read it if payloads contain PII.

T13-Q07 How do you reprocess messages from a DLQ?

Fix the root cause first (code, data, schema). Then run a controlled replay job that reads the DLQ, optionally filters/patches records, and republishes to the original topic (or directly calls the handler), preserving the key for ordering. Replays must be idempotent. Track replay outcomes, and never loop DLQ -> main -> DLQ endlessly (cap with an attempt header).

T13-Q08 What is consumer idempotency and why is it essential?

An idempotent consumer produces the same outcome whether it processes a message once or many times. With at-least-once delivery, duplicates are guaranteed to happen (rebalances, crashes before commit, producer retries without idempotence, replays). Idempotency turns at-least-once into effectively-once.

T13-Q09 Show techniques for implementing an idempotent consumer.

(1) Natural idempotency: `upsert/INSERT ... ON CONFLICT DO NOTHING` keyed by event ID; set-state operations instead of increments. (2) Processed-events table: insert the event ID in the same DB transaction as the business change; a unique-constraint violation means 'already done'. (3) Redis `SET key NX EX ttl` for fast dedupe (weaker, not transactional with DB). (4) Version checks: apply only if `event.version > stored version`.

```
def handle(conn, event):
    with conn:
        # one DB transaction
        cur = conn.cursor()
        cur.execute("""INSERT INTO processed_events(event_id)
                        VALUES (%s) ON CONFLICT DO NOTHING""", (event["event_id"],))
        if cur.rowcount == 0:
            return "duplicate - skipped"
        cur.execute("""UPDATE accounts SET balance = balance + %s
                        WHERE id = %s""", (event["amount"], event["account_id"]))
    return "applied"
```

T13-Q10 Which ID should you deduplicate on: Kafka offset or event ID?

Prefer a producer-assigned business event ID (UUID) carried in the payload or header: it survives replays from a DLQ, re-publishing, and duplicates created by the upstream producer. (topic, partition, offset) only identifies one copy of a record and changes if the same event is published twice.

T13-Q11 What happens to ordering when you use retry topics?

A failed message moves to the retry topic while later messages for the same key continue on the main topic, so they may be processed first. If strict per-key ordering matters (e.g. account balance events), either block and retry in place for that key, park the whole key (keep an 'in-retry' set and route subsequent messages for that key to the retry path too), or design events to be commutative/versioned.

T13-Q12 How do you handle deserialization errors in confluent-kafka-python?

When using `DeserializingConsumer` / the Schema Registry deserializers, errors surface as exceptions (e.g. `ValueDeserializationError` / `ConsumeError`) that carry the raw message. Catch them, send the raw bytes to the DLQ with the error, and advance. With a plain `Consumer` you deserialize manually inside `try/except`.

T13-Q13 How do you prevent a retry storm on a failing downstream?

Use exponential backoff with jitter, cap attempts, add a circuit breaker (stop consuming — `pause()` all partitions — when the error rate crosses a threshold, probe periodically, then `resume()`), apply rate limiting, and alert. Pausing preserves order and avoids flooding DLQs during an outage.

T13-Q14 How do you monitor failure handling in production?

Metrics: consumer lag per partition, processing error rate by error class, retry-topic and DLQ throughput and depth, retry attempt histogram, end-to-end latency. Logs with correlation IDs (event ID in MDC-style context). Alerts on DLQ growth and lag. Dashboards in Grafana/Prometheus, CloudWatch for MSK, or Confluent Control Center.

Worked example: Complete consumer: classify errors -> retry topic or DLQ, idempotent handler

```
import json, logging, traceback
from confluent_kafka import Consumer, Producer

log = logging.getLogger("orders")
class Transient(Exception): pass          # e.g. timeout, 503
class Permanent(Exception): pass          # e.g. validation failure

MAX_INLINE = 3
p = Producer({"bootstrap.servers": "kafka:9092", "enable.idempotence": True})
c = Consumer({"bootstrap.servers": "kafka:9092", "group.id": "orders-svc",
             "enable.auto.commit": False})
c.subscribe(["orders", "orders.retry.1m", "orders.retry.10m"])

def attempt_of(msg):
    return int(dict(msg.headers() or []).get("x-attempt", b"0"))

while True:
    msg = c.poll(1.0)
    if msg is None or msg.error():
        continue
    try:
        event = json.loads(msg.value())      # may raise -> poison pill
        for i in range(MAX_INLINE):          # short inline retries
            try:
                process_idempotently(event)  # dedupe by event_id
                break
            except Transient:
                if i == MAX_INLINE - 1: raise
        except (Transient,) as e:
            route_failure(p, msg, e, attempt_of(msg)) # next retry tier or DLQ
        except (Permanent, json.JSONDecodeError, KeyError) as e:
            p.produce("orders.dlq", key=msg.key(), value=msg.value(),
                    headers=[("x-error", repr(e).encode()),
                            ("x-trace", traceback.format_exc()[-2000:].encode()),
                            ("x-origin", f"{msg.topic():}{msg.partition():}{msg.offset():}".encode())])
    p.poll(0)
    c.commit(message=msg, asynchronous=False)    # move on in every case
```

T14 Schema Management in Kafka (Schema Registry, Avro/Protobuf/JSON Schema)

Kafka stores bytes; schemas give producers and consumers a contract. Know how Schema Registry works, the wire format, subject naming strategies, compatibility modes and how to evolve schemas safely without breaking consumers.

Must-know points

- Schema Registry stores versioned schemas per subject and returns a global schema ID.
- Confluent wire format: magic byte 0 + 4-byte schema ID + serialized payload.
- Subject naming: TopicNameStrategy (default, <topic>-value), RecordNameStrategy, TopicRecordNameStrategy.
- Compatibility: BACKWARD (default), FORWARD, FULL, their _TRANSITIVE forms, NONE.
- Safe evolution: add optional fields with defaults; never change types or reuse field numbers/names.

Interview questions & answers

T14-Q01 Why do we need schema management in Kafka?

Brokers don't validate payloads, so any producer can publish anything and break consumers. A schema defines the structure and types, enables compact binary serialization (Avro/Protobuf), documents the contract between teams, and — with a registry enforcing compatibility — lets schemas evolve without coordinated big-bang deployments.

T14-Q02 What is Confluent Schema Registry and how does it work?

A REST service that stores schemas under subjects with versions, assigns each unique schema a global ID, and checks compatibility on registration. The producer serializer registers or looks up the schema, gets the ID and prepends it to each message. The consumer deserializer reads the ID, fetches (and caches) the writer schema and decodes the payload, resolving it against its own reader schema. Alternatives: AWS Glue Schema Registry, Apicurio, Karapace.

T14-Q03 Describe the Confluent wire format.

Byte 0 = magic byte (0), bytes 1-4 = schema ID as a big-endian int, then the serialized data (for Protobuf also message-index bytes). Only the 5-byte header travels with each record, not the schema itself, so overhead is tiny.

```
import struct
def schema_id_of(raw: bytes) -> int:
    magic, sid = struct.unpack(">BI", raw[:5])
    assert magic == 0, "not Confluent-framed"
    return sid
```

T14-Q04 Avro vs Protobuf vs JSON Schema — how do you choose?

Avro: compact binary, schema always required to read, excellent evolution via defaults, the Kafka/Confluent classic, good for data pipelines and lakes. Protobuf: compact, fast, strong code generation for many languages, great for gRPC-heavy polyglot microservices; evolution via field numbers. JSON Schema: human-readable, easy debugging and browser use, but larger payloads and slower; evolution rules are trickier.

T14-Q05 What is a subject, and what are the subject naming strategies?

A subject is the scope under which schema versions and compatibility are tracked. TopicNameStrategy (default): `<topic>-key / <topic>-value`, one record type per topic. RecordNameStrategy: `<fully.qualified.RecordName>`, allowing multiple event types in one topic with shared evolution across topics. TopicRecordNameStrategy: `<topic>-<RecordName>`, multiple types per topic, evolved per topic.

T14-Q06 Explain the compatibility modes.

BACKWARD (default): consumers using the new schema can read data written with the previous schema — you can delete fields or add fields with defaults; upgrade consumers first. FORWARD: data written with the new schema can be read by consumers using the previous schema — you can add fields or delete optional ones; upgrade producers first. FULL: both. The `_TRANSITIVE` variants check against all previous versions, not just the latest. NONE disables checks.

T14-Q07 Which schema changes are safe under BACKWARD compatibility in Avro?

Safe: add a field WITH a default; remove a field; widen via union with null ('null' first with default null); add an alias. Unsafe: add a field without a default; change a field's type incompatibly (string -> int); rename a field without an alias; remove an enum symbol that old data may contain (unless a default symbol is set).

```
# v1
{"type": "record", "name": "Order", "namespace": "com.shop",
 "fields": [{"name": "id", "type": "string"},
            {"name": "amount", "type": "double"}]}
# v2 - BACKWARD compatible: new optional field with default
{"type": "record", "name": "Order", "namespace": "com.shop",
 "fields": [{"name": "id", "type": "string"},
            {"name": "amount", "type": "double"},
            {"name": "currency", "type": "string", "default": "INR"},
            {"name": "coupon", "type": ["null", "string"], "default": null}]}
```

T14-Q08 What are Protobuf schema evolution rules?

Never change the field number or type of an existing field; don't reuse numbers of deleted fields (mark them **reserved**); adding new fields is safe (old readers ignore unknown fields); in proto3 all scalar fields are optional with zero defaults, so distinguish 'unset' with **optional** or wrapper types; renaming a field is wire-safe (numbers matter) but breaks JSON mapping and generated code.

T14-Q09 Produce Avro messages with confluent-kafka-python and Schema Registry.

Use `SchemaRegistryClient` + `AvroSerializer` with `SerializingProducer` or by calling the serializer manually with a `SerializationContext`.

```
from confluent_kafka import Producer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
from confluent_kafka.serialization import (SerializationContext,
                                           MessageField, StringSerializer)

schema_str = open("order_v2.avsc").read()
sr = SchemaRegistryClient({"url": "http://localhost:8081"})
avro_ser = AvroSerializer(sr, schema_str,
                          conf={"auto.register.schemas": False}) # CI registers
key_ser = StringSerializer("utf_8")
p = Producer({"bootstrap.servers": "localhost:9092"})
order = {"id": "o-1", "amount": 499.0, "currency": "INR", "coupon": None}
p.produce("orders",
          key=key_ser(order["id"]),
          value=avro_ser(order, SerializationContext("orders", MessageField.VALUE)))
p.flush()
```

T14-Q10 Consume Avro messages with the deserializer.

`AvroDeserializer` looks up the writer schema by ID from the registry and decodes; optionally pass your reader schema to project data onto your version.

```
from confluent_kafka import Consumer
from confluent_kafka.schema_registry.avro import AvroDeserializer
avro_de = AvroDeserializer(sr) # writer schema resolved by ID
c = Consumer({"bootstrap.servers": "localhost:9092", "group.id": "billing",
             "auto.offset.reset": "earliest"})
c.subscribe(["orders"])
msg = c.poll(5)
order = avro_de(msg.value(), SerializationContext(msg.topic(), MessageField.VALUE))
print(order["id"], order.get("currency"))
```

T14-Q11 Should producers auto-register schemas in production?

Usually no. Set `auto.register.schemas=false` (and `use.latest.version` as needed) in production and register schemas through CI/CD after a compatibility check (Schema Registry REST `/compatibility` endpoint, Maven/Gradle plugin, or a Python script). This prevents a random service deployment from changing a shared contract.

T14-Q12 How do you check compatibility before deploying?

Call the registry's compatibility API in the pipeline and fail the build if incompatible.

```
import requests, json
schema = open("order_v3.avsc").read()
r = requests.post(
    "http://localhost:8081/compatibility/subjects/orders-value/versions/latest",
    headers={"Content-Type": "application/vnd.schemaregistry.v1+json"},
    data=json.dumps({"schema": schema}))
assert r.json()["is_compatible"], "breaking schema change!"
```

T14-Q13 How do you handle a truly breaking schema change?

Don't force it into the same subject. Create a new topic (e.g. `orders.v2`) or a new record type, dual-write or run a translation stream from v1 to v2 during migration, move consumers over, then retire the old topic. Communicate via a data contract/versioning policy.

T14-Q14 What are data contracts and schema references?

Newer Schema Registry features let a schema carry metadata, tags (e.g. PII), and rules — data-quality rules (CEL), field-level encryption and migration rules — turning the schema into an enforceable data contract. Schema references let one schema import another (shared types like `Address`, or a union of event types in one topic).

Worked example: Register a schema, set compatibility and list versions via the REST API

```
import requests, json
SR = "http://localhost:8081"
H = {"Content-Type": "application/vnd.schemaregistry.v1+json"}
subject = "orders-value"

# 1. enforce BACKWARD_TRANSITIVE for this subject
requests.put(f"{SR}/config/{subject}", headers=H,
             data=json.dumps({"compatibility": "BACKWARD_TRANSITIVE"}))

# 2. register a new version (CI/CD step)
schema = open("order_v2.avsc").read()
r = requests.post(f"{SR}/subjects/{subject}/versions", headers=H,
                 data=json.dumps({"schema": schema, "schemaType": "AVRO"}))
print("schema id", r.json()["id"])

# 3. list versions
print(requests.get(f"{SR}/subjects/{subject}/versions").json())
```

Day-before-interview checklist

- Explain GIL, and when to use threads vs processes vs asyncio, in 60 seconds.
- Write a decorator with arguments (retry/timer) and a context manager from memory.
- Draw the MRO for a diamond and explain what super() calls.
- Sketch a FastAPI app: router, Pydantic model, Depends(), exception handler, lifespan.
- Draw a Kafka cluster: topic, partitions, leader/followers, ISR, consumer group.
- Explain at-most/at-least/exactly-once and show where the commit goes in code.
- Describe the retry-topic + DLQ flow and how the consumer stays idempotent.
- State BACKWARD vs FORWARD compatibility and one safe/unsafe Avro change for each.
- Prepare two stories from your own projects: one scaling/performance problem, one failure you debugged.

Library/API details (Kafka client configs, Schema Registry features, Pandas and Python version behaviour) reflect the state as of 2026; check the official docs for your exact versions before the interview.